

The F2Z polynomial commitment scheme

Relations, reductions, and a normative prover–verifier specification

Normative v1 proof system. F2Z is the polynomial-commitment layer for scalar linear claims about bounded integer or polynomial witnesses whose canonical bits are committed over a binary extension field. A profile-selected ZeroCheck protocol for PESAT or PIMSAT is treated as a black box: it outputs an inner-product claim $\langle \mathbf{v}, \psi(\mathbf{f}) \rangle_R = \mu$, where ψ maps a native witness entry into the scalar evaluation domain. F2Z then

1. rewrites the linear claim over the committed bits;
2. converts bounded integer sums into binary-field grand products;
3. uses GKR to reduce those products to a linear opening claim; and
4. verifies that opening against the original commitment.

This document specifies that PCS reduction, its prover–verifier interaction, its verifier algorithm, and its concrete encoding rules.

1 Overview and interfaces

1.1 The ZeroCheck-to-PCS boundary

Let $\mathbf{x}$ be the public input and let $\mathbf{f}$ be a bounded witness. An admitted source relation expresses its constraints as polynomial equations (PESAT) or as polynomial ideal-membership statements (PIMSAT):

$$\begin{aligned} \text{PESAT} : \quad & p_i(\mathbf{x}, \mathbf{f}) = 0 \quad (\forall i), \\ \text{PIMSAT} : \quad & p_j^{(a)}(\mathbf{x}, \mathbf{f}) \in I_a \quad (\forall a, j). \end{aligned} \tag{1}$$

The polynomial ring, bounded witness domain, and ideals in Equation (1) belong to the source relation. The fixed profile selects that relation and supplies the black-box ZeroCheck contract below. Here “ZeroCheck” names the source-to-linear interface; a PIMSAT instantiation may perform ideal-membership checks internally.

Let M be the native domain of one witness entry and let R be the scalar evaluation domain. ZeroCheck declares how the verifier derives an additive map

$$\psi : M \longrightarrow R, \quad \psi(x + x') = \psi(x) + \psi(x'). \tag{2}$$

For an integer witness, ψ may be reduction into R ; for a polynomial witness, it may combine coefficient reduction with evaluation at a public or transcript-derived point. In either case, $\psi(\mathbf{f})$ means that ψ is applied entry by entry.

ZeroCheck must reduce its source claim to

$$\langle \mathbf{v}, \psi(\mathbf{f}) \rangle_R := \sum_{i=0}^{r-1} v_i \psi(f_i) = \mu, \quad \mathbf{v} \in R^r, \quad \mu \in R. \tag{3}$$

The verifier derives ψ , $\mathbf{v}$, and μ from the ZeroCheck transcript. The same witness $\mathbf{f}$ is retained. Equation (3) is the only source claim consumed by the F2Z core.

1.2 Committing the witness as bits

The ZeroCheck protocol fixes an ordered basis $(\gamma_0, \dots, \gamma_{D_M-1})$ for the native witness domain M . Each γ_j is the j -th public basis element: every accepted f_i has unique coordinates in these elements, and the order of the indices j fixes how those coordinates are serialized. The basis is not computed from the witness or sampled during the protocol. For the concrete integer case, $M = \mathbb{Z}$ and the basis is simply $D_M = 1$, $\gamma_0 = 1$. For a polynomial case with $M = \mathbb{Z}[X]_{<d}$, the usual choice is $D_M = d$ and $\gamma_j = X^j$, ordered from the constant term through X^{d-1} . Every accepted entry then has a canonical, fixed-width decomposition

$$f_i = \sum_{k=0}^{B-1} \sum_{j=0}^{D_M-1} b_{i,k,j} (2^k \gamma_j), \quad b_{i,k,j} \in \{0, 1\}, \quad \mathbf{b} := \text{bits}(\mathbf{f}). \tag{4}$$

The statement fixes the valid bit strings and the order in which the triples (i, k, j) are flattened. Every coefficient vector below uses that same order. F2Z commits to these bits before the first ZeroCheck challenge. Consecutive blocks are packed in an ordered basis of a binary extension field, encoded by the proof system’s linear code, and Merkle committed:

$$P := \text{Pack}(\pi_2(\mathbf{b})), \quad O := \text{Enc}_C(P), \quad \rho := \text{Root}(O). \tag{5}$$

The oracle and root in Equation (5) are carried through ZeroCheck and the F2Z proof. The equality in Equation (3) remains an equality in R ; only the committed representation of the witness lives over the binary field.

1.3 Bitify and the role of GKR

Bitify rewrites the ZeroCheck output directly in terms of the committed bits. The bit $b_{i,k,j}$ inherits the coefficient

$$u_{i,k,j} := v_i \psi(2^k \gamma_j). \tag{6}$$

Substituting the decomposition in Equation (4) into the linear claim gives the following identity. In the concrete proof system, $R = \mathbb{F}_q$ and bits enter R through π_q coordinatewise:

$$\langle \mathbf{v}, \psi(\mathbf{f}) \rangle_R = \mu \iff \langle \pi_q(\mathbf{b}), \mathbf{u} \rangle_R = \mu. \tag{7}$$

This Bitify step sends no message and creates no new commitment.

The remaining problem is to prove the claim in Equation (7), which is evaluated in R , using bits committed over a different field. F2Z tensors the bit vector and lifts one coefficient factor to bounded nonnegative integers λ_i . For a bit matrix B_{bit} , a column sum has the form

$$\mu_c := \sum_i \lambda_i B_{\text{bit}}[c, i] \in \mathbb{Z}. \tag{8}$$

If the sum in Equation (8) lies in an accepted range narrower than the order of $g \in \mathbb{K}^\times$, then exponentiation is injective on that range and

$$g^{\mu_c} = \prod_i g^{\lambda_i B_{\text{bit}}[c, i]} = \prod_i (1 + (g^{\lambda_i} - 1) B_{\text{bit}}[c, i]) \quad \text{in } \mathbb{K}. \tag{9}$$

Equation (9) turns the exact column sums into grand-product claims over the binary field. GKR reduces the batched product computation to a linear opening claim on the same committed bits—an MLE evaluation in the proof system’s fixed specialization. The binary-field opening protocol then checks that terminal claim against O .

2 Instance and parameters

2.1 MLE handoff and concrete proof system

The concrete proof system first flattens the source witness into bounded integer cells. At the PCS boundary it specializes the generic interface as

$$\begin{aligned} r &= 2^{t+s}, & M &= \mathbb{Z}, & \psi &= \pi_q, & D_M &= 1, & \gamma_0 &= 1, & B &= W, \\ f_{c,b} &:= \text{INT}(D(b, c)), & \mathbf{f} &:= (f_{c,b})_{c \in \{0,1\}^s, b \in \{0,1\}^t}. \end{aligned} \quad (10)$$

The order in Equation (10) is c -major, then b -major; the bit order is (c, b, j) . The coefficient at position (c, b) is denoted $v_{c,b}$. This adapter is part of ZeroCheck and must preserve the root from Equation (5).

For $z \in \{0, 1\}^a$ and $\mathbf{r} \in \mathbb{F}_q^a$, define the equality polynomial

$$\text{eq}(z, \mathbf{r}) := \prod_{h=0}^{a-1} ((1 - z_h)(1 - \mathbf{r}_h) + z_h \mathbf{r}_h). \quad (11)$$

The coordinate order in Equation (11) is the order fixed by the tensor layout below.

The concrete proof system instantiates the scalar handoff with an MLE evaluation. Let $D : \{0, 1\}^t \times \{0, 1\}^s \rightarrow [0, 2^W)$ be the committed witness tensor, and let $r_1 \in \mathbb{F}_q^t$ and $r_2 \in \mathbb{F}_q^s$ be transcript-derived. Set

$$\begin{aligned} y &:= \pi_q \circ \widetilde{\text{INT}}(D)(r_1, r_2), \\ w_b &:= \text{can}_q(\text{eq}(b, r_1)), & w'_c &:= \text{eq}(c, r_2). \end{aligned} \quad (12)$$

Expanding the MLE in Equation (12) gives the separable form used by this proof system:

$$y = \sum_{c \in \{0,1\}^s} w'_c \pi_q \left(\sum_{b \in \{0,1\}^t} w_b \text{INT}(D(b, c)) \right). \quad (13)$$

This expansion also fixes the coefficient of each committed bit:

$$u_{c,b,j} := \pi_q(w_b 2^j) w'_c, \quad i := (b \ll w) \mid j. \quad (14)$$

Equations (12)–(14) give the tensor expansion of a single MLE claim. The table records the resulting notation and concrete proof-system bounds.

2.2 Definition table

Object	Exact definition	Type and rule
Domains and ZeroCheck handoff		
Concrete fields (15)	$R = \mathbb{F}_q$, $q = 2^{100} - 15$, $\mathbb{F}_{\text{bin}} = \mathbb{K} = \mathbb{F}_2[X]/(X^{128} + X^7 + X^2 + X + 1) \cong \mathbb{F}_{2^{128}}$.	Linear claims are evaluated in $\mathbb{F}_q$. Packing, grand products, and commitment openings use $\mathbb{K}$.
Witness map	$\mathbf{f} \in \mathcal{W}_B \subseteq M^r$ and $\psi : M \rightarrow R$, with ψ additive as in Equation (2).	M is the native witness domain; $\mathcal{W}_B$ fixes canonical bounds and encodings. ZeroCheck fixes the derivation rule; its verifier derives the instantiated ψ .
ZeroCheck output	$\psi : M \rightarrow R$, $\mathbf{v} \in R^r$, $\mu \in R$, and $\langle \mathbf{v}, \psi(\mathbf{f}) \rangle_R = \mu$.	The verifier derives $(\psi, \mathbf{v}, \mu)$ through the black-box ZeroCheck protocol. See Equation (3).
Concrete adapter	$M = \mathbb{Z}$, $\psi = \pi_q$, $D_M = 1$, $\gamma_0 = 1$, $B = W$, and $f_{c,b} = \text{INT}(D(b, c))$.	Equation (10) fixes the c -major (c, b, j) order and ties the generic witness to the committed tensor.
Public objects	$\text{pp} := (R, M, \mathcal{C}, \Pi_{\text{ZC}}, \Pi_{\text{GKR}}, \Pi_{\text{open}}, \text{transcript}, \text{hash}, \text{core_mode})$, $x_{\text{src}} := (\text{source_statement}, (t, s, W), g)$, $x_{\text{core}} := ((t, s, W), (\mathbf{w}, \mathbf{w}', y), g)$, $\text{com} := (\rho)$.	These public parameters fix ZeroCheck, GKR, wire, transcript, code, and opening algorithms. The ZeroCheck verifier derives x_{core} ; it is not an independent prover input.
Canonical bits	$(\gamma_j)_{j < D_M}$, bit width B , and $\mathbf{b} := \text{bits}(\mathbf{f})$ as in Equation (4).	The proof system fixes the valid bit strings and a single flattening order shared by $\mathbf{b}$ and its coefficient vector.
Canonical maps	$\pi_2 : \mathbb{Z} \rightarrow \mathbb{F}_2$, $\pi_q : \mathbb{Z} \rightarrow \mathbb{F}_q$, and $\text{can}_q : \mathbb{F}_q \rightarrow [0, q)$; for $0 \leq v < 128$, $\beta_v := X^v \bmod (X^{128} + X^7 + X^2 + X + 1)$.	can_q returns the unique integer representative used for bounded exponent lifts. $(\beta_v)_{v < 128}$ is the ordered polynomial basis used by Pack.
Tensor layout and commitment		
Shape and cells	$\mathbf{b} := \text{bits}(\mathbf{f})$, $W = 2^w$, and $D(b, c) \in [0, 2^W)$, $d := t + w$, $m := d + s$, $m_p := m - 7$.	(t, s, W) is public, $\mathbf{f}$ is bounded, the bit-decomposition is canonical, and $(b, c) \in \{0, 1\}^t \times \{0, 1\}^s$.
Bit tensor (16)	$B_{\text{bit}}[c][[(b \ll w) \mid j]] := \text{bit}_j(D(b, c))$, $\text{INT}(D(b, c)) := \sum_{j=0}^{W-1} 2^j B_{\text{bit}}[c][[(b \ll w) \mid j]]$.	$B_{\text{bit}} \in \mathbb{F}_2^{2^s \times 2^d}$, $0 \leq j < W$, and $\mathbf{b} = \text{vec}(B_{\text{bit}})$ has 2^m bits.
Pack and commit (17)	$P[c2^{d-7} + i_{\text{hi}}] := \sum_{v=0}^{127} B_{\text{bit}}[c][[(i_{\text{hi}} \ll 7) \mid v] \beta_v]$, $O := \text{Enc}_{\mathcal{C}}(P)$, $\rho := \text{Root}(O)$.	$0 \leq i_{\text{hi}} < 2^{d-7}$, $0 \leq v < 128$, and $P \in \mathbb{K}^{2^m p}$. The seven low index bits select the 128 coefficients of the fixed ordered $\mathbb{F}_2$ -basis of $\mathbb{K}$.
MLE-derived core claim		
Core tuple	$(\mathbf{w}, \mathbf{w}', y)$, $ \mathbf{w} = 2^t$, $ \mathbf{w}' = 2^s$, $y = \mu$.	$0 \leq w_b < 2^{\text{qbits}}$ is an integer; $w'_c, y \in \mathbb{F}_q$. This is the separable tensor form of one linear claim.

Object	Exact definition	Type and rule
Definition table (continued)		
Bit coefficient	$u_{c,b,j} := \pi_q(w_b 2^j) w'_c, \quad i := (b \ll w) \mid j.$	$0 \leq i < 2^d$ and $u_{c,b,j} \in \mathbb{F}_q$. The verifier derives this coefficient using Equation (14); it is never supplied.
Chunking and admissibility		
Chunk decomposition (18)	Parameters. $q_{\text{bits}} := 100, \quad c_w := 127 - t - W, \quad L := \max(\lceil q_{\text{bits}}/c_w \rceil, 1).$ Digits. $\omega_b^{(\ell)} := \lfloor w_b/2^{c_w \ell} \rfloor \bmod 2^{c_w}, \quad w_b = \sum_{\ell=0}^{L-1} 2^{c_w \ell} \omega_b^{(\ell)}.$ Columns. $u_c^{(\ell)} := \sum_b \omega_b^{(\ell)} \text{INT}(D(b, c)).$	$0 \leq \ell < L$; the admissibility rows below imply $c_w > 0$. $0 \leq \omega_b^{(\ell)} < 2^{c_w}.$ $0 \leq u_c^{(\ell)} < 2^{c_w+t+W} = 2^{127}.$
Shape and interface (19)	$W \in 2^{\mathbb{N}}, \quad d \geq 7, \quad t + W \leq 126, \quad 22 \leq m \leq 35.$	Require every implied dimension, canonical public values, and the correct claim variant.
Generator (20)	$g \in \mathbb{K}^\times, \quad \text{ord}(g) = 2^{128} - 1,$ $\text{ord}(g) = 2^{128} - 1 \iff g^{(2^{128}-1)/p} \neq 1 \text{ for every prime } p \mid 2^{128} - 1.$	Together with $u_c^{(\ell)} < 2^{127}$, full order makes $u \mapsto g^u$ injective over every accepted fold.

Equations (16) and (17) fix the committed coordinate order. Equations (18)–(20) fix the admitted chunk schedule and the range on which exponentiation is injective.

3 Relations, reductions, and proof objects

3.1 How to read this section

Section 1 described the main F2Z reduction informally. This section names its intermediate claims and the reductions between them. The reductions appear in dependency order.

Throughout the direct path, the encoded oracle O and its public root ρ remain fixed. What changes is the public claim attached to that committed data. Bitify also changes the witness view from the native entries $\mathbf{f}$ to their canonical bits $\mathbf{b} = \text{bits}(\mathbf{f})$.

An encoded relation contains triples $(x, O; \text{wit})$: x is the explicit instance, O is the implicit encoded oracle, and wit is the witness. For a public witness encoder E_{wit} , the binding condition is

$$O = \text{Enc}_C(E_{\text{wit}}(\text{wit})). \quad (21)$$

Merkle compilation publishes the public root $\rho := \text{Root}(O)$ for this oracle; ρ is not a proof field. Every relation below is additionally parameterized by the fixed public context $\chi := (\text{pp}, t, s, W, g)$ and its derived dimensions. The context is omitted from displayed instances and is carried unchanged by every edge. The three protocol interfaces used below are $\Pi_{\text{ZC}}, \Pi_{\text{red}} := \Pi_{\text{GKR}},$ and $\Pi_{\text{io}} := \Pi_{\text{open}}.$

The edge notation is

$$\begin{aligned}
\mathcal{R}_0 &\equiv_F \mathcal{R}_1 && \text{denotes deterministic retyping,} \\
\mathcal{R}_0 &\xrightarrow{F} \mathcal{R}_1 && \text{denotes deterministic compilation,} \\
\mathcal{R}_0 &\xRightarrow{\Pi} \mathcal{R}_1 && \text{denotes an interactive reduction,} \\
\mathcal{R}_0 &\dashrightarrow \mathcal{R}_1 && \text{denotes a conditional extension or classification only.}
\end{aligned} \quad (22)$$

A proof fragment lists transmitted values only; challenges, successor coefficients, exponentiated values, batching coefficients, and work tables are verifier-derived.

3.2 ZeroCheck black-box handoff

The reduction chain starts with ZeroCheck. Let $\mathcal{R}_{\text{src}}$ be the PESAT or PIMSAT source relation selected by the fixed profile. F2Z treats its ZeroCheck protocol as a black box and assumes only the handoff below. The deterministic **Encode** step binds the witness to the encoded oracle:

$$\begin{aligned}
\mathcal{R}_{\text{src}}^{\text{enc}} &:= \left\{ (x_{\text{src}}, O; \mathbf{f}) : (x_{\text{src}}, \mathbf{f}) \in \mathcal{R}_{\text{src}}, \quad \mathbf{f} \in \mathcal{W}_B, \right. \\
&\quad \left. O = \text{Enc}_C(\text{Pack}(\pi_2(\text{bits}(\mathbf{f})))) \right\}.
\end{aligned} \quad (23)$$

This encoded relation is the sole source consumed by the PCS.

ZeroCheck supplies one black-box reduction

$$\mathcal{R}_{\text{src}}^{\text{enc}} \xrightarrow{\Pi_{\text{ZC}}} \mathcal{R}_{\text{Lin-ModFieldBin}}. \quad (24)$$

The proof fragment π_{ZC} and all transcript events on this edge belong to ZeroCheck. Its verifier derives $(\psi, \mathbf{v}, \mu)$ and the required separable tuple $(\mathbf{w}, \mathbf{w}', y)$ with $y = \mu$. The adapter must satisfy Equation (10) and $v_{c,b} = \pi_q(w_b) w'_c$, while retaining the same witness, oracle O , and root ρ . An oracle mismatch or incomplete event ledger causes rejection. Once this reduction succeeds, the source constraints have been discharged; the F2Z core receives only the derived linear claim. For a PIMSAT source, ZeroCheck may perform ideal-membership checks internally.

3.3 The Lin-ModFieldBin relation

The ZeroCheck handoff leaves a scalar linear claim about native witness entries. The Lin-ModFieldBin relation records that claim together with the same encoded oracle. Its public ordered basis and statement-bound injection bits $\mathcal{W}_B \hookrightarrow \{0, 1\}^n$ give the fixed reconstruction

$$f_i = \sum_{k < B} \sum_{j < D_M} b_{i,k,j} 2^k \gamma_j, \quad b_{i,k,j} \in \{0, 1\}, \quad \mathbf{b} = \text{bits}(\mathbf{f}). \quad (25)$$

This equation fixes both reconstruction and bit order. The additive map $\psi : M \rightarrow R$ is instantiated by ZeroCheck and carried explicitly in the public successor instance.

The resulting module relation is

$$\begin{aligned}
\mathcal{R}_{\text{Lin-ModFieldBin}} &:= \left\{ ((\psi, \mathbf{v}, \mu), O; \mathbf{f}) : \mathbf{f} \in \mathcal{W}_B, \quad \psi : M \rightarrow R \text{ is ZeroCheck-admitted and additive,} \quad \mathbf{v} \in R^r, \quad \mu \in R, \quad O = \text{Enc}_C(\text{Pack}(\pi_2(\text{bits}(\mathbf{f})))) \right. \\
&\quad \left. \sum_i v_i \psi(f_i) = \mu \text{ in } R \right\}.
\end{aligned} \quad (26)$$

This is the exact source-to-core seam. The first core reduction simply rewrites the claim using the canonical bits already bound to O .

Direct reductions.

- **Reduction 1: Deterministic Bitify.**

$$\mathcal{R}_{\text{Lin-ModFieldBin}} \equiv_{\text{Bitify}} \mathcal{R}_{\text{Lin-BitsDual}}, \quad u_{i,k,j} := v_i \psi(2^k \gamma_j). \quad (27)$$

Bitify transmits nothing: it retypes the witness as $\mathbf{b} := \text{bits}(\mathbf{f})$ and derives the public coefficients. Since O already binds these canonical bits, no new Booleanity test is needed.

3.4 The Lin-BitsDual relation

After Bitify, the claim is expressed directly on the committed bits. The corresponding core relation is

$$\mathcal{R}_{\text{Lin-BitsDual}} := \left\{ ((\mathbf{u}, \mu), O; \mathbf{b}) : \mathbf{b} \in \text{im}(\text{bits}) \subseteq \{0, 1\}^n, \quad \mathbf{u} \in R^n, \quad \mu \in R, \quad O = \text{Enc}_C(\text{Pack}(\pi_2(\mathbf{b}))), \right. \\ \left. \langle \pi_q(\mathbf{b}), \mathbf{u} \rangle_R = \mu \right\}. \quad (28)$$

Here $\pi_q(\mathbf{b})$ denotes the coordinatewise bit embedding into $R = \mathbb{F}_q$. Restricting $\mathbf{b}$ to $\text{im}(\text{bits})$ makes Bitify an equivalence rather than a one-way reduction. Let $\iota_{2 \rightarrow \mathbb{K}} : \mathbb{F}_2 \hookrightarrow \mathbb{K}$ denote the corresponding coordinatewise embedding into the binary field. The claim now concerns the committed bits, but it is still evaluated in $R = \mathbb{F}_q$, whereas the oracle proof operates over $\mathbb{K}$. To cross this boundary, reshape the bits as $B_{\text{bit}}[c, i]$ and write the public coefficient in the same c -major order as $u_{c,i} = u_c^{(2)} u_i^{(1)}$. With canonical integer representatives λ_i of $u_i^{(1)}$, the prover exposes the column values

$$\mu_c := \sum_i \lambda_i B_{\text{bit}}[c, i] \in \mathbb{Z}. \quad (29)$$

The verifier first checks that these values reconstruct the original scalar claim:

$$\sum_c u_c^{(2)} \pi_q(\mu_c) = \mu \quad \text{in } R = \mathbb{F}_q. \quad (30)$$

It then derives $y_i := g^{\lambda_i}$ and $\nu_c := g^{\mu_c}$. The bounded integer sums become the binary-field product claims

$$\prod_i (1 + (y_i - 1) B_{\text{bit}}[c, i]) = \nu_c \quad (\forall c). \quad (31)$$

These are the product claims consumed by Π_1 .

Direct reductions.

- **Reduction 1: Produce the F2Z encoded PESAT claim.**

$$\mathcal{R}_{\text{Lin-BitsDual}} \xRightarrow{\Pi_1} \mathcal{R}_{\text{PESAT}}(\mathbb{K}, Q_{\mathbb{F}_2^{\mathbb{Z}}}, \mathcal{C}, \text{Pack}). \quad (32)$$

- **Proof:** The prover sends $\pi_1 = (\mu_c)_c$. The verifier checks Equation (30) before accepting the successor.
- **Result:** the verifier derives $\mathbf{y} = (g^{\lambda_i})_i$, $\boldsymbol{\nu} = (g^{\mu_c})_c$, and the PESAT instance; the bit witness and O are unchanged.
- **Scope:** the first core F2Z reduction.
- **Proof-system restriction: weighted coefficient shape.** The chunked mod- q proof system admits only coefficients of the form

$$u_{c,b,j} = \pi_q(w_b 2^j) w'_c, \quad 0 \leq w_b < 2^{q_{\text{bits}}}, \quad w'_c \in \mathbb{F}_q. \quad (33)$$

This weighted form is the admissible public coefficient shape.

3.4.1 Active-chunk instantiation of Π_1

The fixed mod- q proof system implements the same Π_1 reduction in chunks, keeping every exponent inside the injective range of g . For the admitted weighted shape, write

$$w_b = \sum_{\ell=0}^{L-1} 2^{c w_\ell} \omega_b^{(\ell)}, \quad u_c^{(\ell)} := \sum_b \omega_b^{(\ell)} \text{INT}(D(b, c)). \quad (34)$$

The verifier derives $\mathcal{A} := \{\ell < L : \exists b, \omega_b^{(\ell)} \neq 0\}$ from the public weights, requires $\mathcal{A} \neq \emptyset$, and processes its elements in increasing order. The values in Equation (34) determine the proof fragment

$$\pi_1^{\text{chunk}} := \left(\nu^{(\ell)}, u^{(\ell)} \right)_{\ell \in \mathcal{A} \uparrow}, \quad \begin{aligned} u^{(\ell)} &= (u_c^{(\ell)})_{c \in \{0,1\}^s}, \\ \nu^{(\ell)} &= (g^{u_c^{(\ell)}})_{c \in \{0,1\}^s}. \end{aligned} \quad (35)$$

This tuple lists exactly the transmitted chunk fields. For each active chunk, $\nu^{(\ell)}$ is a NARG record and $u^{(\ell)}$ is a checked, non-absorbed hint. Inactive chunks have the derived value $u^{(\ell)} = 0$. The verifier rejects a missing, duplicate, out-of-order, or noncanonical active pair; requires $0 \leq u_c^{(\ell)} < 2^{127}$; and checks $g^{u_c^{(\ell)}} = \nu_c^{(\ell)}$ before its next squeeze. Because $\text{ord}(g) = 2^{128} - 1$, the stated range makes the accepted exponent unique. The checked hints reconstruct the original column values:

$$\mu_c := \sum_{\ell=0}^{L-1} 2^{c w_\ell} u_c^{(\ell)} = \sum_b w_b \text{INT}(D(b, c)). \quad (36)$$

After receiving the final active vector, the verifier checks

$$\sum_c w'_c \pi_q(\mu_c) = y. \quad (37)$$

This chunked reconstruction discharges the scalar claim before the opening phase.

For each active ℓ , the verifier also derives

$$i = (b \ll w) \mid j, \quad \omega_i^{(\ell)} = \omega_b^{(\ell)} 2^j, \quad y_i^{(\ell)} = g^{\omega_i^{(\ell)}}. \quad (38)$$

These exponents define one successor PESAT claim for each active chunk:

$$\bigwedge_{\ell \in \mathcal{A}} \mathcal{R}_{\text{PESAT}}(\mathbb{K}, Q_{\mathbb{F}_2^{\mathbb{Z}}}^{(\ell)}, \mathcal{C}, \text{Pack}). \quad (39)$$

3.5 The encoded F2Z PESAT relation

Each accepted chunk asserts one grand-product identity. For one active chunk, write its residual polynomial as

$$Q_c^{(\ell)}(B_{\text{bit}}) := \prod_i (1 + (y_i^{(\ell)} - 1) B_{\text{bit}}[c, i]) - \nu_c^{(\ell)}. \quad (40)$$

The encoded F2Z PESAT relation collects the equations $Q_c^{(\ell)}(B_{\text{bit}}) = 0$ for every column:

$$\mathcal{R}_{\text{PESAT}}(\mathbb{K}, Q_{\mathbb{F}_2^{\mathbb{Z}}}^{(\ell)}, \mathcal{C}, \text{Pack}) \\ := \left\{ ((\mathbf{y}^{(\ell)}, \boldsymbol{\nu}^{(\ell)}), O; B_{\text{bit}}) : B_{\text{bit}} \in \mathbb{F}_2^{2^s \times 2^d}, \quad O = \text{Enc}_C(\text{Pack}(B_{\text{bit}})), \quad Q_c^{(\ell)}(B_{\text{bit}}) = 0 \ (\forall c) \right\}. \quad (41)$$

Omitting ℓ gives the unchunked notation $\mathcal{R}_{\text{PESAT}}(\mathbb{K}, Q_{\mathbb{F}_2^{\mathbb{Z}}}, \mathcal{C}, \text{Pack})$. Thus the grand-product claim is represented by this encoded PESAT relation. Checking all products directly would be expensive. GKR instead compresses their verification into a linear claim on the same committed bits.

Direct reductions and classifications.

• Reduction 1: Linearize with GKR.

$$\mathcal{R}_{\text{PESAT}}(\mathbb{K}, Q_{\mathbb{F}_2\mathbb{Z}}, \mathcal{C}, \text{Pack}) \xrightarrow{\Pi_{\text{red}}} \mathcal{R}_{\text{Lin}}(\mathbb{K}, \mathcal{C}, \text{Pack}). \quad (42)$$

For the chunked family, this reduction runs once per $\ell \in \mathcal{A}$.

- **Input:** the chunk's PESAT instance, the unchanged root $\rho = \text{Root}(O)$, and the transcript after $\zeta^{(\ell)}$ and $e_0^{(\ell)}$; call this bundle $\mathcal{I}_{\text{GKR}}^{(\ell)}$.
- **Proof:**

$$(\text{tr}', p_{t_\ell}, \mu_\ell) \leftarrow \text{GKR.Verify}(\text{pp}_{\text{GKR}}, \mathcal{I}_{\text{GKR}}^{(\ell)}, \pi_{\text{red}}^{(\ell)}; \text{tr}). \quad (43)$$

This call consumes exactly one canonical GKR fragment $\pi_{\text{red}}^{(\ell)}$ and returns the updated transcript tr' . The GKR protocol fixes its messages, challenge order, encodings, tags, and rejection rules.

- **Result:** the verifier derives $(\mathbf{a}_\ell, \mu_\ell)$ for $\mathcal{R}_{\text{Lin}}(\mathbb{K}, \mathcal{C}, \text{Pack})$, retaining the same O and bit witness. In the current proof system, with $B_{\text{bit}, \mathbb{K}} := \iota_{2 \rightarrow \mathbb{K}}(B_{\text{bit}})$ coordinatewise,

$$\mathbf{a}_\ell[z] = \text{eq}(z, p_{t_\ell}), \quad \widetilde{B_{\text{bit}, \mathbb{K}}}(p_{t_\ell}) = \mu_\ell. \quad (44)$$

- **Scope:** this edge assumes a GKR instantiation with a byte-exact transcript and a proof of the reduction. Its error is $\varepsilon_{\text{red}}^{(\ell)}$ per active chunk.
- **Classification: Degree-zero, zero-ideal PIMSAT.** With $S = \mathbb{K}[X]$, $S_{\leq 0} = \mathbb{K}$, and $I = (0)$, membership means exactly $Q_c^{(\ell)} = 0$. Boolean witnesses keep their bit domain; unrestricted $\mathbb{K}$ variables also require $Z^2 - Z = 0$.

3.6 The encoded LIN relation

After GKR, neither the source constraints nor the grand products remain. The encoded LIN relation records the sole remaining linear claim. Using the coordinatewise embedding defined above, put $\mathbf{b}_{\mathbb{K}} := \iota_{2 \rightarrow \mathbb{K}}(\pi_2(\mathbf{b}))$:

$$\mathcal{R}_{\text{Lin}}(\mathbb{K}, \mathcal{C}, \text{Pack}) := \left\{ ((\mathbf{a}, \mu), O; \mathbf{b}) : \mathbf{b} \in \{0, 1\}^n, \quad \mathbf{a} \in \mathbb{K}^n, \quad \mu \in \mathbb{K}, \right. \\ \left. O = \text{Enc}_{\mathcal{C}}(\text{Pack}(\pi_2(\mathbf{b}))), \quad \langle \mathbf{a}, \mathbf{b}_{\mathbb{K}} \rangle_{\mathbb{K}} = \mu \right\}. \quad (45)$$

This is the linear target consumed by the opening IOP.

For the fixed geometry $n = 2^m$, write $\tilde{a}$ and $\tilde{b}_{\mathbb{K}}$ for the multilinear extensions of $\mathbf{a}$ and $\mathbf{b}_{\mathbb{K}}$. The direct GKR successor is the special case $a[z] = \text{eq}(z, r)$, so its claim is already $\tilde{b}_{\mathbb{K}}(r) = \mu$. A general coefficient need not have this form. The final reduction now opens the derived linear claim against the original root.

Direct reductions.

• Reduction 1: Run the linear-relation IOP.

$$\mathcal{R}_{\text{Lin}}(\mathbb{K}, \mathcal{C}, \text{Pack}) \xrightarrow{\Pi_{\text{iopp}}} \mathcal{R}_{\text{triv}}. \quad (46)$$

- **Proof:** In Equation (46), π_{iopp} proves the derived $(\mathbf{a}, \mu)$ claim against the public root $\rho = \text{Root}(O)$.
- **Result:** acceptance or rejection; no public relation remains.
- **Scope:** the final direct edge. The construction below gives one concrete implementation.

3.6.1 Fixed opening protocol's internal expansion of Π_{iopp}

The fixed recursive-opening protocol implements Π_{iopp} as follows. Direct MLE-shaped claims proceed to the ring switch, while virtual claims first pass through the LinCheck below.

1. **One LinCheck for a general coefficient.** The direct mode skips this item. For a general claim $\langle \mathbf{a}, \mathbf{b}_{\mathbb{K}} \rangle_{\mathbb{K}} = \mu$, the prover and verifier run the degree-two sumcheck for

$$\sum_{x \in \text{bits}^m} \tilde{a}(x) \tilde{b}_{\mathbb{K}}(x) = \mu. \quad (47)$$

Starting with $\sigma_{-1} := \mu$, round i sends one degree-two polynomial g_i and enforces

$$g_i(0) + g_i(1) = \sigma_{i-1}, \quad r_i \leftarrow \mathbb{K}, \quad \sigma_i := g_i(r_i), \quad 0 \leq i < m. \quad (48)$$

These equations fix every round check and challenge deadline. Let $r = (r_{\text{lo}}, r_{\text{hi}})$, where $|r_{\text{lo}}| = 7$. The first ring-switch message derives

$$z := \sum_{v=0}^{127} \text{eq}(v, r_{\text{lo}}) s_v = \tilde{b}_{\mathbb{K}}(r), \quad \sigma_{m-1} = \tilde{a}(r) z. \quad (49)$$

The verifier checks Equation (49) immediately after absorbing that message and before the next squeeze. This terminal rule uses no division and has no exceptional branch when $\tilde{a}(r) = 0$. The LinCheck proof fragment is $\pi_{\text{LC}} := (g_i)_{i=0}^{m-1}$ and its soundness error is at most $\varepsilon_{\text{LC}} := 2m/|\mathbb{K}|$. After the check, the same s vector is reused by the ring switch; no second LinCheck or extra evaluation target is introduced.

2. **Per-claim ring switch.** For $0 \leq v < 128$, define the v -th bit plane of a packed row by $B_{\text{bit}, v}[c, i_{\text{hi}}] := B_{\text{bit}}[c][i_{\text{hi}} \ll 7 \mid v]$, using the coordinate order from Equation (17). For each active ℓ , the prover transmits

$$\mathbf{s}^{(\ell)} = (s_v^{(\ell)})_{v < 128}, \quad \mathbf{s}_v^{(\ell)} := \widetilde{B_{\text{bit}, v}}(r_{\text{hi}}^{(\ell)}). \quad (50)$$

After all vectors in Equation (50) are absorbed, the verifier squeezes one shared r'' and derives a packed claim

$$O = \text{Enc}_{\mathcal{C}}(P), \quad \langle B_\ell, P \rangle_{\mathbb{K}} = \beta_\ell. \quad (51)$$

In Equation (51), the witness is $P = \text{Pack}(\pi_2(\mathbf{b}))$ and $\rho = \text{Root}(O)$ is unchanged. This step has error $\varepsilon_{\text{ring}}^{(\ell)}$. For a direct MLE-shaped source, the verifier checks $z = \mu_\ell$; for the single general claim normalized by LinCheck, it checks Equation (49). Thus the ring switch itself still consumes only one MLE evaluation.

3. **Batch.** The verifier squeezes fresh $\eta \in \mathbb{K}^{|\mathcal{A}|}$ and derives, without a proof field,

$$B := \sum_{\ell \in \mathcal{A}} \eta_\ell B_\ell, \quad \beta := \sum_{\ell \in \mathcal{A}} \eta_\ell \beta_\ell. \quad (52)$$

The batch values in Equation (52) define the resulting internal claim $O = \text{Enc}_{\mathcal{C}}(P)$ and $\langle B, P \rangle_{\mathbb{K}} = \beta$, with batching error $\varepsilon_{\text{batch}}$.

4. **Recursive opening.** The prover transmits $\pi_{\text{open}} = \pi_{\text{lig}}$, one fixed recursive constrained-code proof containing its internal proximity messages and authenticated openings. The verifier derives all opening challenges and accepts only if the proof verifies against (ρ, B, β) . This completes Π_{iopp} with error $\varepsilon_{\text{open}}$.

3.7 The virtual Lin-BitsDual extension

The virtual extension provides an optional route into the same LIN relation when the claimed values come from a public linear transformation of the committed bits.

For a public $\mathbb{F}_2$ -linear map $T : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^{n'}$ and $h_0 \in \mathbb{F}_2^{n'}$, let $\text{lift}_2 : \mathbb{F}_2 \rightarrow \{0, 1\} \subset \mathbb{Z}$ act coordinatewise and put $\bar{h} = h_0 + T(\pi_2(\mathbf{b}))$. Define

$$\mathcal{R}_{\text{Lin-BitsDual}}^{\text{virt}} := \left\{ ((T, \mathbf{a}_q, \mu, h_0), O; \mathbf{b}) : \mathbf{b} \in \{0, 1\}^n, \quad \mathbf{a}_q \in \mathbb{F}_q^{n'}, \quad \mu \in \mathbb{F}_q, \right. \\ \left. O = \text{Enc}_{\mathcal{C}}(\text{Pack}(\pi_2(\mathbf{b}))), \quad \langle \pi_q(\text{lift}_2(\bar{h})), \mathbf{a}_q \rangle_{\mathbb{F}_q} = \mu \right\}. \quad (53)$$

This is the source relation for the virtual branch.

Direct reductions.

• Conditional virtual reduction.

$$\mathcal{R}_{\text{Lin-BitsDual}}^{\text{virt}} \xrightarrow{\Pi_{\text{partial-core}}} \mathcal{R}_{h\text{-Lin}}^{\text{virt}}. \quad (54)$$

- **Proof:** For the conditional edge in Equation (54), π_{virtual} carries the chosen partial-core and GKR messages; the virtual table is not transmitted.
- **Result:** the verifier derives $(\mathbf{a}_{\mathbb{K}}, \mu')$.
- **Scope:** available only after the virtual mode defines the proof format, transcript, verifier, and error bound.

Let $h_{0,\mathbb{K}} := \iota_{2 \rightarrow \mathbb{K}}(h_0)$ and let $T_{\mathbb{K}}$ be the scalar extension of T . Only $\mathbf{b}$ is committed. The virtual linear seam is

$$\mathcal{R}_{h\text{-Lin}}^{\text{virt}} := \left\{ ((T, h_0, \mathbf{a}_{\mathbb{K}}, \mu'), O; \mathbf{b}) : \mathbf{b} \in \{0, 1\}^n, \quad \mathbf{a}_{\mathbb{K}} \in \mathbb{K}^{n'}, \quad \mu' \in \mathbb{K}, \right. \\ \left. O = \text{Enc}_{\mathcal{C}}(\text{Pack}(\pi_2(\mathbf{b}))), \right. \\ \left. \langle \mathbf{a}_{\mathbb{K}}, h_{0,\mathbb{K}} + T_{\mathbb{K}}(\mathbf{b}_{\mathbb{K}}) \rangle_{\mathbb{K}} = \mu' \right\}. \quad (55)$$

This relation is the partial core's required successor. Its public adjoint rewrite transmits nothing:

$$\mathcal{R}_{h\text{-Lin}}^{\text{virt}} \equiv \text{Adj}_{T_{\mathbb{K}}}^* \mathcal{R}_{\text{Lin}}(\mathbb{K}, \mathcal{C}, \text{Pack}), \quad (56)$$

$$\mathbf{a}_b := T_{\mathbb{K}}^*(\mathbf{a}_{\mathbb{K}}), \quad \mu_b := \mu' - \langle \mathbf{a}_{\mathbb{K}}, h_{0,\mathbb{K}} \rangle_{\mathbb{K}}.$$

The exact rewrite in Equation (56) need not preserve the direct mode's MLE coefficient shape. The virtual mode therefore invokes the LinCheck in Equations (47)–(49) before the existing ring switch. It must provide a deterministic evaluator for the adjoint coefficient, for example by the dual identity

$$\widetilde{T_{\mathbb{K}}^*}(\mathbf{a}_{\mathbb{K}})(r) = \langle \mathbf{a}_{\mathbb{K}}, T_{\mathbb{K}}(\text{eq}(\cdot, r)) \rangle_{\mathbb{K}}, \quad (57)$$

The virtual profile must prove that this evaluator equals the declared full adjoint. No dense adjoint vector and no multi-evaluation ring adapter are required. A virtual profile is admissible only if it specifies the partial-core predecessor, evaluator, LinCheck ledger, and composed error. Otherwise the verifier rejects; direct mode always rejects this source.

3.8 The complete relation graph and proof object

Collecting the preceding reductions gives the complete relation graph. Along the direct path, Bitify changes the witness view once, from $\mathbf{f}$ to $\mathbf{b}$, while the encoded oracle and root remain fixed. Let $\mathcal{R}_{\text{triv}} := \{(\perp, \perp; \perp)\}$:

$$\begin{aligned} \mathcal{R}_{\text{src}} &\xrightarrow{\text{Encode}} \mathcal{R}_{\text{src}}^{\text{enc}}, \\ \mathcal{R}_{\text{src}}^{\text{enc}} &\xrightarrow{\Pi_{\text{ZC}}} \mathcal{R}_{\text{Lin-ModFieldBin}}, \\ \mathcal{R}_{\text{Lin-ModFieldBin}} &\equiv \text{Bitify} \mathcal{R}_{\text{Lin-BitsDual}} \xrightarrow{\Pi_1} \mathcal{R}_{\text{PESAT}}(\mathbb{K}, Q_{\mathbb{F}_2\mathbb{Z}}, \mathcal{C}, \text{Pack}) \\ \mathcal{R}_{\text{PESAT}}(\mathbb{K}, Q_{\mathbb{F}_2\mathbb{Z}}, \mathcal{C}, \text{Pack}) &\xrightarrow{\Pi_{\text{red}}} \mathcal{R}_{\text{Lin}}(\mathbb{K}, \mathcal{C}, \text{Pack}) \xrightarrow{\Pi_{\text{iopp}}} \mathcal{R}_{\text{triv}}. \end{aligned} \quad (58)$$

Encode and Bitify add no proof fields; chunking and recursive opening refine the displayed edges. Concatenating the messages carried by the interactive edges gives the semantic proof object

$$\pi_{\text{F2Z}} = \left(\pi_{\text{ZC}}, \left(\nu^{(\ell)}, u^{(\ell)}, \pi_{\text{red}}^{(\ell)} \right)_{\ell \in \mathcal{A} \text{ increasing}}, \pi_{\text{LC}}, (s^{(\ell)})_{\ell \in \mathcal{A} \text{ increasing}}, \pi_{\text{lig}} \right). \quad (59)$$

The fields in Equation (59) are replayed according to Section 4. Public inputs are external to the proof; other omitted values are derived. The v1 wire format partitions transmitted fields, in replay order, as

$$\pi_{\text{wire}} := (N_{\text{F2Z}}, H_{\text{F2Z}}), \quad \begin{aligned} \text{narglen}(\pi) &:= |N_{\text{F2Z}}|, & \text{hintlen}(\pi) &:= |H_{\text{F2Z}}|, \\ \text{payloadlen}(\pi) &:= |N_{\text{F2Z}}| + |H_{\text{F2Z}}|, & \text{wirelen}(\pi) &:= |\text{HostEncode}_{\text{v1}}(N_{\text{F2Z}}, H_{\text{F2Z}})|. \end{aligned} \quad (60)$$

Benchmarks MUST report all four lengths. Section 6.4 fixes their byte meaning, the outer 40-byte container, and the exact record counts and exhaustion rules. With ZeroCheck error ε_{ZC} , $\varepsilon_{\text{red}}^{(\ell)}$, the optional ε_{LC} , $\varepsilon_{\text{ring}}^{(\ell)}$, $\varepsilon_{\text{batch}}$, and $\varepsilon_{\text{open}}$,

$$\varepsilon_{\text{F2Z}} \leq \varepsilon_{\text{ZC}} + \sum_{\ell \in \mathcal{A}} (\varepsilon_{\text{red}}^{(\ell)} + \varepsilon_{\text{ring}}^{(\ell)}) + \varepsilon_{\text{LC}} + \varepsilon_{\text{batch}} + \varepsilon_{\text{open}}. \quad (61)$$

Equation (61) is the proof system's composed soundness bound. The direct mode sets $\pi_{\text{LC}} = ()$ and $\varepsilon_{\text{LC}} = 0$.

4 Prover–verifier interaction

4.1 Transcript channels and schedule

Equation (60) defines the two proof streams; these badges remain normative without color.

Badge	Bytes	Verifier action
N[x]	NARG	Consume a canonical typed record and absorb that same encoding before any dependent squeeze; PoW alone uses the checked-deferred read in Section 6.6.
H[x]	Hint	Consume, never absorb, and check against the bound prefix before the next squeeze.
D[x]	None	Public, verifier-derived, or challenge value; no proof bytes, with the exact framing and squeeze transition of Section 6.6.
N/H[π]	Both	The subordinate ledger fixes each field's channel and both replay orders.

[John: WTF is a verifyStream? Do we really need that. It's not defined on the overleaf.] Object notation $\text{Verify}(\dots, \pi; \text{tr})$ denotes the equivalent $\text{VerifyStream}(\dots; \text{tr}, c_N, c_H)$. It follows the subordinate ledger over the fixed proof system's bounded input slices, performs the same checks and events, and succeeds only at both declared cursor endpoints.

[John: WTF is this relation of CheckHint? Is it really needed?] Given public data and bound NARG prefix A , a hint h is admissible only if its accepted value is computationally unique:

$$\Pr[\exists h \neq h' : \text{CheckHint}_i(A, h) = \text{CheckHint}_i(A, h') = 1] \leq \text{negl}(\lambda). \quad (62)$$

The uniqueness condition in Equation (62) MUST be checked before the next Squeeze. Acceptance requires no pending hint and exact exhaustion of both cursors; missing, misordered, or trailing records reject.

[John: This mother fucking thing can just be said more simply of how we are doing a transcript absorb. It can simply be absorb(domain|length of bytes [bytes] and then we can use squeeze. Just define the goddamn transcript as a sponge] Let $\text{Frame}_{\text{v1}}(\tau, \ell, k, i; z)$ be the byte frame of Equation (79). Before the first proof challenge,

$$\text{tr}_0 := \text{Absorb}(\text{Absorb}(\text{tr}_{\text{init}}, \text{Frame}_{\text{v1}}(0x0001, \perp_{32}, \perp_{32}, \perp_{32}; \text{DomainV1})), \text{Frame}_{\text{v1}}(0x0002, \perp_{32}, \perp_{32}, \perp_{32}; \text{StatementV1})). \quad (63)$$

The payloads in Equation (63) are fixed by Equation (81). Later absorbs are domain-separated by their numeric tag and fixed (ℓ, k, i) scope fields. A squeeze updates tr and yields a derived, non-transmitted value; each populated interaction row denotes one event. The tables realize Equation (58) as matched prover/verifier steps in this exact order:

$$\begin{aligned}
 & (P_1/V_1); (P_2/V_2); (P_3/V_3); \\
 & \text{for } \ell \in \mathcal{A} \text{ in increasing order : } ((P_4/V_4)^{(\ell)}; (P_5/V_5)^{(\ell)}); \\
 & (P_6/V_6); (P_7/V_7); (P_8/V_8); (P_9/V_9); (P_{10}/V_{10}); (P_{11}/V_{11}).
 \end{aligned} \tag{64}$$

Equation (64) is the normative top-level replay order. Right ledgers contain post-step claims; *internal* claims and superscript- (ℓ) copies add no paper relation.

P1/V1. Commit and bind the source statement

Prover P	Traffic	Verifier V
$\mathbf{b} := \text{bits}(\mathbf{f}),$ Construct $P := \text{Pack}(\pi_2(\mathbf{b})),$ $O := \text{Enc}_C(P), \quad \rho := \text{Root}(O).$	$\xrightarrow{\text{D}[\text{com}]}$	Validate the public parameters, statement, shape, code, and root type; then set $\text{tr} \leftarrow \text{Absorb}_{\text{statement}}(\text{tr}, (\text{pp}, x_{\text{src}}, \text{com})).$

REDUCTION LEDGER

INITIALIZATION Encode
FROM $\mathcal{R}_{\text{src}}$
TO $\mathcal{R}_{\text{src}}^{\text{enc}}$

CURRENT STATE

| SRC **RELATION** **NEW**
 $\mathcal{R}_{\text{src}}^{\text{enc}}(x_{\text{src}}, O; \mathbf{f})$

WHERE

$\rho \quad \rho = \text{Root}(O)$

P2/V2. Run ZeroCheck

The fixed ZeroCheck protocol owns this entire reduction and retains the same encoded oracle.

Prover P	Traffic	Verifier V
Construct $(\text{tr}, \pi_{\text{ZC}}) \leftarrow \text{ZeroCheck.Prove}(\text{pp}, x_{\text{src}}, \mathbf{f}; \text{tr}).$	$\xrightarrow{\text{N}/\text{H}[\pi_{\text{ZC}}]}$	Compute $(\text{tr}, \mathcal{I}_{\text{LMFB}}, \mathbf{w}, \mathbf{w}') \leftarrow \text{VerifyZeroCheck}(x_{\text{src}}, \rho, \pi_{\text{ZC}}; \text{tr}),$ derive $(\psi, \mathbf{v}, \mu) := \text{Claim}(\mathcal{I}_{\text{LMFB}}),$ set $y := \mu,$ require $\text{SameRoot}(\mathcal{I}_{\text{LMFB}}, \rho) = 1,$ $\psi = \pi_q,$ the concrete adapter of Equation (10), $v_{c,b} = \pi_q(w_b)w'_c,$ and all public bounds; then bind the derived core instance before its first challenge.

REDUCTION LEDGER

RELATION REDUCTION Π_{ZC}
FROM $\mathcal{R}_{\text{src}}^{\text{enc}}$
TO $\mathcal{R}_{\text{Lin-ModFieldBin}}$

CURRENT STATE

| LMFB **RELATION** **NEW**
 $\langle \mathbf{v}, \psi(\mathbf{f}) \rangle_R = \mu$

WHERE

$\rho \quad \rho = \text{Root}(O)$

P3/V3. Bitify the weighted linear relation

Prover P	Traffic	Verifier V
Retain $\mathbf{b} = \text{bits}(\mathbf{f})$ and its matrix form $B_{\text{bit}}.$		Derive, without a message or challenge, $u_{c,b,j} := v_{c,b} \pi_q(2^j) = \pi_q(w_b 2^j) w'_c.$ This is retying by semilinearity, not a Booleanity round. Encode CoreSeamV1 by Equation (82) and absorb its tag-0003 frame at scope $(\perp_{32}, \perp_{32}, \perp_{32})$ before the first chunk record.

REDUCTION LEDGER

DETERMINISTIC REWRITE Bitify
FROM $\mathcal{R}_{\text{Lin-ModFieldBin}}$
TO $\mathcal{R}_{\text{Lin-BitsDual}}$

CURRENT STATE

| LBD **RELATION** **NEW**
 $\langle \pi_q(\mathbf{b}), \mathbf{u} \rangle_R = y$

WHERE

$\rho \quad \rho = \text{Root}(O)$

P4/V4. Run chunk ℓ of Π_1

Prover P	Traffic	Verifier V
Before sending anything, compute $u_c^{(\ell)} = \sum_i \omega_i^{(\ell)} B_{\text{bit}}[c, i], \quad \nu_c^{(\ell)} = g^{u_c^{(\ell)}}.$	$\xrightarrow{\text{N}[\nu^{(\ell)}]}$ $\xrightarrow{\text{H}[u^{(\ell)}]}$ $\xleftarrow{\text{D}[\zeta^{(\ell)}]}$	Read and absorb the canonical typed vector $\nu^{(\ell)}.$ Before any later squeeze, require $ u^{(\ell)} = 2^s, \quad 0 \leq u_c^{(\ell)} < 2^{127},$ and $g^{u_c^{(\ell)}} = \nu_c^{(\ell)}$ for every $c.$ The order of g and the range bound make this accepted exponent unique. Then set $\zeta^{(\ell)} \leftarrow \text{Squeeze}_{\mathbb{K}}^s(\text{tr}),$ $e_0^{(\ell)} := \nu^{(\ell)}(\zeta^{(\ell)}).$

REDUCTION LEDGER

PAPER-EDGE FRAGMENT $\Pi_1^{(\ell)}$
FROM $\mathcal{R}_{\text{Lin-BitsDual}}$
TO $\mathcal{R}_{\text{PESAT}}(\mathbb{K}, Q_{\mathbb{Z}, \mathbb{Z}}^{(\ell)}, C, \text{Pack})$

CURRENT STATE

| LBD **RELATION** **USED**
 $\langle \pi_q(\mathbf{b}), \mathbf{u} \rangle_R = y$
| PESAT **RELATION** **NEW**
 $Q_c^{(\ell)}(B_{\text{bit}}) = 0 \quad (\forall c)$

WHERE

$\rho \quad \rho = \text{Root}(O)$
 $P \quad P = \text{Pack}(\pi_2(\mathbf{b}))$

P5/V5. Run the GKR reduction for chunk ℓ

The PCS-level schedule treats $\Pi_{\text{red}}^{(\ell)}$ as the named GKR subprotocol. Its input is the current PESAT claim, its unchanged commitment $\rho = \text{Root}(O)$, and the transcript state after $\zeta^{(\ell)}$ and $e_0^{(\ell)}$ are derived. Denote this typed input bundle by $\mathcal{I}_{\text{GKR}}^{(\ell)}$. Its output must have the MLE-shaped coefficient $\mathbf{a}_\ell[z] = \text{eq}(z, pt_\ell)$ and satisfy $\widetilde{B_{\text{bit}, \mathbb{K}}}(pt_\ell) = \mu_\ell$ against the same root.

Prover P	Traffic	Verifier V
Construct the fixed GKR protocol's canonical proof fragment $\pi_{\text{red}}^{(\ell)} \leftarrow \text{GKR.Prove}(\text{ppGKR}, \mathcal{I}_{\text{GKR}}^{(\ell)}, B_{\text{bit}}; \text{tr})$.	$\xrightarrow{\text{N}[\text{H}[\pi_{\text{red}}^{(\ell)}]]}$	Compute $(\text{tr}', pt_\ell, \mu_\ell) \leftarrow \text{GKR.Verify}(\text{ppGKR}, \mathcal{I}_{\text{GKR}}^{(\ell)}, \pi_{\text{red}}^{(\ell)}; \text{tr})$, require success and complete consumption of the GKR fragment, then set $\text{tr} \leftarrow \text{tr}'$.

Retain the witness value $\widetilde{B_{\text{bit}, \mathbb{K}}}(pt_\ell)$.

All internal GKR messages and challenges are owned by the fixed GKR protocol and use its domain namespace. Treating GKR atomically here does not permit an implementation to omit that subordinate transcript and encoding specification.

For the merged-forest GKR protocol, the repeated phase- x value $\text{H}[S_{x,k}^{(\ell)}]$ is a valid hint because the prefix already fixes $e_k^{(\ell)}$ and the verifier immediately requires $S_{x,k}^{(\ell)} = e_k^{(\ell)}$ before any following squeeze. A canonical implementation SHOULD omit that redundant field and mark it $\text{D}[e_k^{(\ell)}]$ instead. The phase- c claimed sums, sumcheck messages, and terminal child pairs remain $\text{N}[\text{GKR messages}]$: their consistency checks do not uniquely determine them and later challenges must bind them. The outer encoded field $\text{presums}[\ell].\text{claimed_sums}$ is distinct from those phase- c claims. Wire format v1 omits it and supplies the derived value $\text{D}[[e_d^{(\ell)} + 1]]$ to the MultiDegree pre-sumcheck. A decoder that encounters the reserved legacy tag 3081 MUST reject.

P6/V6. Complete Π_1 's reconstruction check

Prover P	Traffic	Verifier V
		After all active chunks, set inactive $u^{(\ell)} := 0$, reconstruct $\mu_c^{\text{col}} := \sum_{\ell=0}^{L-1} 2^{c \cdot w_\ell} u_c^{(\ell)} \in \mathbb{Z}$, and require the paper's tensor reconstruction equality $\sum_c w'_c \pi_q(\mu_c^{\text{col}}) = y$. This check has no message, absorb, or challenge.

P7/V7. Apply the virtual adjoint when needed

The direct mode skips this optional P7/V7 step and carries its current LIN family unchanged. The virtual mode substitutes its specified partial core for P4/V4 through P6/V6, producing $h = h_{0,\mathbb{K}} + T_K(b_K)$ and $\langle a_h, h \rangle_{\mathbb{K}} = \mu_h$ by the proof-carrying conditional edge

$$\mathcal{R}_{\text{Lin-BitsDual}}^{\text{virt}} \xrightarrow{\Pi_{\text{partial-core}}} \mathcal{R}_{h\text{-Lin}}^{\text{virt}}. \quad (65)$$

Equation (65) is the required proof-carrying predecessor; only the following adjoint rewrite is zero-message:

$$a_b := T_K^* a_h, \quad \mu_b := \mu_h - \langle a_h, h_{0,\mathbb{K}} \rangle_{\mathbb{K}}, \quad \langle a_b, b_K \rangle_{\mathbb{K}} = \mu_b. \quad (66)$$

Equation (66) generates no message or challenge. A virtual-mode implementation without this predecessor and the LinCheck evaluator of Equation (57) rejects virtual inputs.

REDUCTION LEDGER

GKR SUBPROTOCOL $\Pi_{\text{red}}^{(\ell)}$
FROM $\mathcal{R}_{\text{PESAT}}(\mathbb{K}, Q_{\text{FZ}}^{(\ell)}, \mathcal{C}, \text{Pack})$
TO $\mathcal{R}_{\text{Lin}}^{(\ell)}(\mathbb{K}, \mathcal{C}, \text{Pack})$

CURRENT STATE

LBD **RELATION**
 $\langle \pi_q(b), u \rangle_R = y$
LIN[Pack] **RELATION** **NEW**
 $\widetilde{B_{\text{bit}, \mathbb{K}}}(pt_r) = \mu_r$
 $(r \in \mathcal{A}, r \leq \ell)$

WHERE

$\rho \quad \rho = \text{Root}(O)$
 $P \quad P = \text{Pack}(\pi_2(b))$

REDUCTION LEDGER

PAPER COMPOSITION $\Pi_1; \Pi_{\text{red}}$
FROM $\mathcal{R}_{\text{Lin-BitsDual}}$
TO $\bigwedge_{\ell \in \mathcal{A}} \mathcal{R}_{\text{Lin}}^{(\ell)}(\mathbb{K}, \mathcal{C}, \text{Pack})$

CURRENT STATE

LIN[Pack] **RELATION** **REDUCED**
 $\widetilde{B_{\text{bit}, \mathbb{K}}}(pt_\ell) = \mu_\ell$
 $(\ell \in \mathcal{A})$

WHERE

$\rho \quad \rho = \text{Root}(O)$

REDUCTION LEDGER

OPTIONAL ZERO-MESSAGE BRANCH $\text{Adj}_{T_K^*}$
FROM $\mathcal{R}_{h\text{-Lin}}^{\text{virt}}$
TO $\mathcal{R}_{\text{Lin}}(\mathbb{K}, \mathcal{C}, \text{Pack})$

CURRENT STATE

LIN[Pack] **RELATION** **USED**
 $\widetilde{B_{\text{bit}, \mathbb{K}}}(pt_\ell) = \mu_\ell$
 $(\ell \in \mathcal{A})$

WHERE

$\rho \quad \rho = \text{Root}(O)$

P8/V8. LinCheck a general coefficient when needed

The direct mode skips P8/V8 because every GKR successor is already MLE-shaped. The virtual mode runs exactly one degree-two LinCheck on the general claim produced by P7/V7. Write $g_i(X) = \gamma_{i,0} + \gamma_{i,1}X + \gamma_{i,2}X^2$. For $i = 0, \dots, m-1$, P8 sends its three canonical coefficients under NARG tag 4002 and scope $(\perp_{32}, \perp_{32}, i)$. V8 absorbs that frame, requires

$$g_i(0) + g_i(1) = \sigma_{i-1}, \quad \sigma_{-1} = \mu_b, \quad (67)$$

then squeezes $r_i \in \mathbb{K}$ under tag 4103 at the same scope and sets $\sigma_i = g_i(r_i)$. Equation (67) is checked before every dependent squeeze. Put $r = (r_{lo}, r_{hi})$, $|r_{lo}| = 7$. P8/V8 leaves the terminal gate

$$\sigma_{m-1} = \tilde{a}_b(r) \tilde{b}_{\mathbb{K}}(r), \quad (68)$$

Equation (68) is checked by P9/V9 with its first ring vector. No other LinCheck is permitted.

Prover P	Traffic	Verifier V
Send each degree-two g_i in increasing i .	$\xrightarrow{N[(g_i)_{i < m}]}$	Check, absorb, and squeeze each round in exact alternation; derive (r, σ_{m-1}) and the public evaluator $r \mapsto \tilde{a}_b(r)$.

P9/V9. Ring-switch each active linear claim

For the direct mode, split $pt_\ell = (r_{lo}^{(\ell)}, r_{hi}^{(\ell)})$ with $|r_{lo}^{(\ell)}| = 7$. For the virtual singleton, use $\ell = 0$ and the LinCheck point r instead. In both cases define

$$z_\ell := \sum_{v=0}^{127} \text{eq}(v, r_{lo}^{(\ell)}) s_v^{(\ell)}, \quad \begin{cases} z_\ell = \mu_\ell, & \text{direct MLE source,} \\ \sigma_{m-1} = \tilde{a}_b(r) z_0, & \text{virtual LinCheck source.} \end{cases} \quad (69)$$

The applicable gate in Equation (69) is checked after the NARG frame is absorbed and before tag 4101; no division occurs. After the shared challenge r'' , the successor coefficients are

$$\begin{aligned} \bar{s}_u^{(\ell)} &:= \sum_v [s_v^{(\ell)}]_u \beta_v, & \Phi_{r''}(z) &:= \sum_u \text{eq}(u, r'') [z]_u, \\ \beta_\ell &:= \sum_u \text{eq}(u, r'') \bar{s}_u^{(\ell)}, & B_\ell(Y) &:= \Phi_{r''}(\text{eq}(r_{hi}^{(\ell)}, Y)). \end{aligned} \quad (70)$$

Prover P	Traffic	Verifier V
For every $\ell \in \mathcal{A}$ and $0 \leq v < 128$, compute $s_v^{(\ell)} := \widetilde{B_{\text{bit},v}}(r_{hi}^{(\ell)})$.	$\xrightarrow{N[s^{(\ell)}]}$	For each ℓ , validate and absorb the canonical $s^{(\ell)}$ frame, then require the applicable equality in Equation (69) before any squeeze. After the last vector squeeze $r'' \leftarrow \text{Squeeze}_{\mathbb{K}}^7(\text{tr})$.
With $[z]_u$ denoting coordinate u in the fixed $\mathbb{K}/\mathbb{F}_2$ basis, derive (B_ℓ, β_ℓ) by Equation (70).	$\xleftarrow{D[r'']}$	Independently derive the same $(B_\ell, \beta_\ell)_{\ell \in \mathcal{A}}$.

P10/V10. Batch the packed linear relations

Prover P	Traffic	Verifier V
Form $B := \sum_{\ell \in \mathcal{A}} \eta_\ell B_\ell$, $\beta := \sum_{\ell \in \mathcal{A}} \eta_\ell \beta_\ell$.	$\xleftarrow{D[\eta]}$	For each active ℓ in increasing order, squeeze $\eta_\ell \in \mathbb{K}$ under tag 4102 and scope $(\ell, \perp_{32}, \perp_{32})$. Independently derive the same (B, β) .

REDUCTION LEDGER

Π_{IOPP} INTERNAL Π_{LC}
FROM $\mathcal{R}_{\text{Lin}}(\mathbb{K}, \mathcal{C}, \text{Pack})$
TO $\text{EvalGate}(\mathbb{K}, \mathcal{C}, \text{Pack})$

CURRENT STATE

LIN[Pack] RELATION USED
 $\widehat{B_{\text{bit},\mathbb{K}}}(pt_\ell) = \mu_\ell$
 $(\ell \in \mathcal{A})$

WHERE

$\rho \quad \rho = \text{Root}(O)$

REDUCTION LEDGER

Π_{IOPP} INTERNAL Π_{ring}^A
FROM $\bigwedge_{\ell \in \mathcal{A}} \mathcal{R}_{\text{Lin}}^{(\ell)}(\mathbb{K}, \mathcal{C}, \text{Pack})$
TO $\bigwedge_{\ell \in \mathcal{A}} \mathcal{R}_{\text{Lin}}^{(\ell)}(\mathbb{K}, \mathcal{C}, \text{id})$

CURRENT STATE

LIN[id] RELATION NEW
 $\langle B_\ell, P \rangle_{\mathbb{K}} = \beta_\ell$
 $(\ell \in \mathcal{A})$

WHERE

$\rho \quad \rho = \text{Root}(O)$
 $P \quad P = \text{Pack}(\pi_2(b))$

REDUCTION LEDGER

Π_{IOPP} INTERNAL Batch_η
FROM $\bigwedge_{\ell \in \mathcal{A}} \mathcal{R}_{\text{Lin}}^{(\ell)}(\mathbb{K}, \mathcal{C}, \text{id})$
TO $\mathcal{R}_{\text{Lin}}(\mathbb{K}, \mathcal{C}, \text{id})$

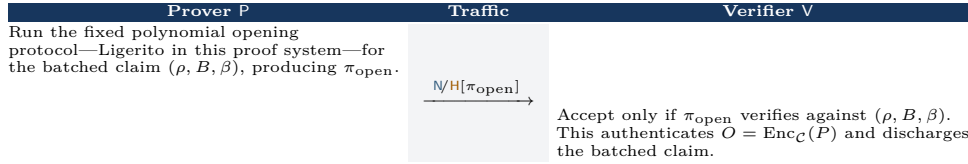
CURRENT STATE

LIN[id] RELATION NEW
 $\langle B, P \rangle_{\mathbb{K}} = \beta$

WHERE

$\rho \quad \rho = \text{Root}(O)$
 $P \quad P = \text{Pack}(\pi_2(b))$

P11/V11. Open the batched claim



REDUCTION LEDGER

Π_{IOPP} INTERNAL Π_{open}
FROM $\mathcal{R}_{\text{Lin}}(\mathbb{K}, C, \text{id})$
TO $\mathcal{R}_{\text{triv}}$

CURRENT STATE
nothing left to prove

Recursive-opening channel ledger. For the recursive Ligerito protocol, the following partition is normative at the field level. The NARG records are replayed in verifier-event order, not Rust structure-field order; hint records are consumed at the opening level where their checks appear.

Class	Fields	Binding and verification deadline
$N[\cdot]$	Initial and recursive roots; sum-check messages; OOD values; <code>final_proof.yr</code> ; query- and fold-grinding nonces.	Each is observed or checked-and-absorbed before a dependent challenge. PoW nonces remain NARG because multiple values may satisfy the predicate; Section 6.7 fixes their tags and schedule.
$H[\text{open}_0]$	<code>initial_proof.opened_rows</code> and <code>initial_proof.merkle_proof</code> .	The prior initial root and sampled query positions bind the rows and path. Merkle verification must succeed before the next sumcheck-introduction challenge.
$H[\text{open}_i]$	Each recursive proof's <code>opened_rows</code> and <code>merkle_proof</code> .	Verify against the already absorbed preceding-level root and sampled queries before the next batching challenge. Consume levels in increasing recursive order.
$H[\text{open}_{\text{fin}}]$	<code>final_proof.opened_rows</code> and <code>final_proof.merkle_proof</code> .	Verify against the preceding root after the final queries are fixed and before the final batching challenge or acceptance.

The authenticated rows are hints because the earlier Merkle root computationally fixes their accepted contents; the authentication nodes only certify that fixed opening. The whole π_{open} MUST NOT be placed on the hint channel. A recursive-opening implementation must reject before any later squeeze on a failed row-length or Merkle check and must exhaust both substreams before returning success. Section 6.8 fixes the byte payload, scope, cardinality, and exact replay position of every field summarized in this table.

The Rust type `FlockCommitHint` is unrelated terminology: it is a prover-local commitment cache/key, not an H_{F2Z} record. Its rows, packed columns, message, and prover data are not proof bytes; only the public commitment/root enters the verifier-facing protocol. Steps P8/V8 through P11/V11 are the fixed recursive-opening protocol's internal expansion of the paper edge Π_{IOPP} . The verifier accepts iff every displayed check succeeds and the proof parser consumes exactly the fields specified by these steps.

5 Verifier algorithm

The verifier rejects on any failed gate or helper, malformed or unsupported value, or cursor mismatch. NARG and hint cursors are independent; a hint stays pending until its predicate succeeds, and every squeeze requires `NoPendingHint`. Records and prover/verifier steps are processed in increasing order. The labels below remain meaningful without color.

[N] read / bind P1/V1–P3/V3 statement	[H] read / check P4/V4–P5/V5 per chunk	[S] challenge P6/V6–P7/V7 reconstruction	[C] reject gate P8/V8–P11/V11 opening	[D] local value P8/V8–P11/V11 opening	[CALL] subprotocol
--	---	---	---	---	---

Procedure VerifyF2Z.

Require: $\text{pp}, x_{\text{src}}, \text{com} = (\rho), \pi_{\text{host}}$

Ensure: true or false

P1/V1: Validate and bind the commitment

- $$\mathcal{R}_{\text{src}} \xrightarrow{\text{Encode}} \mathcal{R}_{\text{src}}^{\text{enc}}$$
- 1: $(N_{F2Z}, H_{F2Z}, n_N, n_H, b_N, b_H) \leftarrow \text{DecodeHostV1}(\pi_{\text{host}})$
 - 2: [C] **Require** $[\text{HeaderV1} = 1 \wedge \text{wire_version} = 1]$
 - 3: [C] **Require** $[\text{ValidatePublic}(\text{pp}, x_{\text{src}}, \rho) = 1]$
 - 4: Read (t, s, W, g) from x_{src} ; [C] **Require** $[t, s \in \mathbb{N} \wedge W \in 2^{\mathbb{N}}]$
 - 5: [D] $w \leftarrow \log_2 W$, $d \leftarrow t + w$, $m \leftarrow d + s$, $m_p \leftarrow m - 7$, $c_w \leftarrow 127 - t - W$
 - 6: [C] **Require** $[d \geq 7 \wedge t + W \leq 126 \wedge 22 \leq m \leq 35 \wedge c_w > 0]$
 - 7: [C] **Require** $[\text{ord}(g) = 2^{128} - 1]$
 - 8: $\text{tr} \leftarrow$ [N] **Absorb** $(\text{tr}_{\text{init}}, \text{Frame}_{v1}(0x0001, \perp_{32}, \perp_{32}, \perp_{32}; \text{DomainV1}))$ using Equation (81)
 - 9: $\text{tr} \leftarrow$ [N] **Absorb** $(\text{tr}, \text{Frame}_{v1}(0x0002, \perp_{32}, \perp_{32}, \perp_{32}; \text{StatementV1}))$ using Equation (81)
 - 10: Initialize independent cursors [D] $c_N \leftarrow \text{Cursor}(N_{F2Z}, n_N, b_N)$, $c_H \leftarrow \text{Cursor}(H_{F2Z}, n_H, b_H)$, and $\text{PendingHint} \leftarrow 0$
 - 11: Load the fixed ZeroCheck and GKR event ledgers and the canonical constrained-code construction from pp

P2/V2: Verify ZeroCheck

- $$\mathcal{R}_{\text{src}}^{\text{enc}} \xrightarrow{\Pi_{\text{ZC}}} \mathcal{R}_{\text{Lin-ModFieldBin}}$$
- 12: $(\text{tr}, \mathcal{I}_{\text{LMFB}}, w, w') \leftarrow$ [CALL] **VerifyZeroCheckStream** $(\text{pp}, x_{\text{src}}, \rho; \text{tr}, c_N, c_H)$
 - 13: [D] $(\psi, v, \mu) \leftarrow \text{Claim}(\mathcal{I}_{\text{LMFB}})$
 - 14: [C] **Require** $[\text{SameRoot}(\mathcal{I}_{\text{LMFB}}, \rho) = 1 \wedge \psi = \pi_q \wedge \text{ConcreteLMFBShape}(\mathcal{I}_{\text{LMFB}}, t, s, W) = 1]$
 - 15: [D] $y \leftarrow \mu$
 - 16: [C] **Require** $[|w| = 2^t \wedge |w'| = 2^s \wedge 0 \leq w_b < 2^{100} \wedge w'_c, y \in \mathbb{F}_q]$

P3/V3: Apply deterministic Bitify

- $$\mathcal{R}_{\text{Lin-ModFieldBin}} \equiv \text{Bitify } \mathcal{R}_{\text{Lin-BitsDual}}$$
- 17: [D] $\mathcal{I}_{\text{LBD}} \leftarrow \text{Bitify}(\mathcal{I}_{\text{LMFB}})$
 - 18: [C] **Require** $[\text{WeightShape}(\mathcal{I}_{\text{LBD}}, w, w', y) = 1]$
 - $u_{c,b,j} = \pi_q(w_b 2^j) w'_c$
 - 19: [D] $x_{\text{core}} \leftarrow ((t, s, W), (w, w', y), g)$
 - 20: $\text{tr} \leftarrow$ [N] **Absorb** $(\text{tr}, \text{Frame}_{v1}(0x0003, \perp_{32}, \perp_{32}, \perp_{32}; \text{CoreSeamV1}))$ using Equation (82)
 - 21: [D] $L \leftarrow \max(\lceil 100/c_w \rceil, 1)$
 - 22: [D] $\omega_b^{(\ell)} \leftarrow \lfloor w_b / 2^{c_w \ell} \rfloor \bmod 2^{c_w}$
 - 23: [D] $\mathcal{A} \leftarrow \{\ell < L : \exists b, \omega_b^{(\ell)} \neq 0\}$; [C] **Require** $[\mathcal{A} \neq \emptyset]$
 - 24: Set $u^{(\ell)} \leftarrow 0^{2^s}$ for $\ell \notin \mathcal{A}$
 - 25: Fix, without advancing either cursor, the remaining semantic schedule $((\nu^{(\ell)}, u^{(\ell)}, \pi_{\text{red}}^{(\ell)})_{\ell \in \mathcal{A}^\uparrow}, \pi_{\text{LC}}, (s^{(\ell)})_{\ell \in \mathcal{A}^\uparrow}, \pi_{\text{open}})$
 - 26: **for** $\ell \in \mathcal{A}$ in increasing order **do**

P4/V4: Verify chunk ℓ of Π_1

- $$\mathcal{R}_{\text{Lin-BitsDual}} \xrightarrow{\Pi_1^{(\ell)}} \mathcal{R}_{\text{PESAT}}(\mathbb{K}, Q_{\mathbb{F}_{2^2}}^{(\ell)}, C, \text{Pack})$$
- 27: $(\nu^{(\ell)}, F_\nu) \leftarrow$ [N] **ReadNarg** $(c_N, 0x2001, \ell, \perp_{32}, \perp_{32})$

```

28:  tr ← [N] Absorb(tr, Fν)
29:  PendingHint ← 1, (u(ℓ), Fu) ← [H] ReadHint(cH, 0x2081, ℓ, ⊥32, ⊥32)
30:  [C] Require[|u(ℓ)| = 2s ∧ 0 ≤ uc(ℓ) < 2127 ∧ (guc(ℓ))c = ν(ℓ)]
31:  PendingHint ← 0
32:  [D] ωi(ℓ) ← ωb(ℓ) 2j, yi(ℓ) ← gωi(ℓ), where i = (b ≪ w) | j
33:  ζ(ℓ) ← [S] Squeezeℝ, 0x2101s(tr; ℓ), e0(ℓ) ← ν(ℓ)(ζ(ℓ))

```

P5/V5: Verify the GKR reduction

```


$$\mathcal{R}_{\text{PESAT}}(\mathbb{K}, Q_{\mathbb{F}_2^2}^{(\ell)}, \mathcal{C}, \text{Pack}) \xrightarrow[\text{red}]{\Pi^{(\ell)}} \mathcal{R}_{\text{Lin}}^{(\ell)}(\mathbb{K}, \mathcal{C}, \text{Pack})$$

34:  [D] IGKR(ℓ) ← GKRIInput(xcore, ρ, y(ℓ), ν(ℓ), ζ(ℓ), e0(ℓ))
35:  (okGKR, tr, ptℓ, μℓ) ← [CALL] GKR.VerifyStream(ppGKR, TGKR(ℓ); tr, cN, cH)
36:  [C] Require[okGKR = 1]
37:  end for
P6/V6: Complete the Π1 reconstruction
38:  [D] μc ← ∑ℓ=0L-1 2cwℓ uc(ℓ) over ℤ
39:  [C] Require[∑c wc πq(μc) = y]
P7/V7: Normalize the optional virtual source

$$\mathcal{R}_{h\text{-Lin}}^{\text{virt}} \equiv \text{Adj}_{T^*} \mathcal{R}_{\text{Lin}}(\mathbb{K}, \mathcal{C}, \text{Pack})$$

40:  [C] Require[core_mode = direct]
The virtual mode uses a specified partial-core, adjoint, and the one-LinCheck adapter below.
P8/V8: LinCheck a general coefficient when required
41:  [C] Require[core_mode = direct ⇒ πLC = () ∧ εLC = 0]
The virtual mode alternates tag-4002 degree-two messages and tag-4103 challenges, then checks its terminal gate with the first tag-4001 vector.
P9/V9: Verify the ring switch
internal expansion of Πiopp
42:  for ℓ ∈ A in increasing order do
43:    Write ptℓ = (rlo(ℓ), rhi(ℓ))
44:    (s(ℓ), Fs) ← [N] ReadNarg(cN, 0x4001, ℓ, ⊥32, ⊥32)
45:    [C] Require[|rlo(ℓ)| = 7 ∧ |s(ℓ)| = 128]
46:    tr ← [N] Absorb(tr, Fs)
47:    [C] Require[∑v=0127 eq(v, rlo(ℓ)) sv(ℓ) = μℓ]
48:  end for
49:  r'' ← [S] Squeezeℝ, 0x41017(tr)
50:  for ℓ ∈ A in increasing order do
51:    [D] s̄u(ℓ) ← ∑v [sv(ℓ)]u βv, Φr''(z) ← ∑u eq(u, r'') [z]u
52:    [D] βℓ ← ∑u eq(u, r'') s̄u(ℓ), Bℓ(Y) ← Φr''(eq(rhi(ℓ), Y))
53:  end for
P10/V10: Batch the packed LIN claims
internal expansion of Πiopp
54:  for ℓ ∈ A in increasing order do
55:    ηℓ ← [S] Squeezeℝ, 0x4102(tr; ℓ, ⊥32, ⊥32)
56:  end for
57:  [D] B ← ∑ℓ ∈ A ηℓ Bℓ, β ← ∑ℓ ∈ A ηℓ βℓ
P11/V11: Open the batched claim
fixed polynomial opening (Ligerito in this proof system)
58:  [C] Require[B = DerivedCoefficient(xsrc, tr) ∧ dom(B) = {0, 1}mp]
59:  tr ← [N] Absorb(tr, Framev1(0x5001, ⊥32, ⊥32, ⊥32; LE32(mp) || Encℝ(β)))
60:  [C] Require[[CALL] RecVerifyStream(pp, ρ, B, β; tr, cN, cH) = 1]
61:  [C] Require[NoPendingHint = 1 ∧ NargEOF(cN) = 1 ∧ HintEOF(cH) = 1 ∧ HostEOF(πhost) = 1 ∧ Counts(cN, cH) = (nN, nH)]
62:  return true

```

The helper verifiers own the checks, absorptions, and challenges specified in Section 4; they do not introduce additional paper-level relation nodes. They also own their subordinate channel ledgers: every NARG read advances only c_N, every hint read advances only c_H, and a helper MUST NOT return success with an unchecked hint. P4/V4's N[ν^(ℓ)], H[u^(ℓ)] order is realized by tags 2001 and 2081. Every read, absorb, squeeze, PoW check, and EOF condition uses the byte rules in Section 6; no helper may fall back to structure-field order or an opaque serialization.

6 Wire and transcript format v1

Normative v1 proof system. This section fixes the bytes consumed by a conforming prover and verifier. The proof system has two physical proof streams, prefix-free typed records, a BLAKE3 transcript, SHA-256 Merkle roots, and the recursive Ligerito and proof-of-work parameters below. A proof using the legacy positional/bincode codec is not a v1 proof.

6.1 Version and fixed algorithms

The eight-byte container magic is the ASCII string F2ZPCS\0\0, and the wire version is one. Changing any normative rule in this section requires a new wire version.

The transcript and its XOF are unkeyed BLAKE3; Merkle nodes and roots use SHA-256; PoW uses unkeyed BLAKE3-256. The field basis is the polynomial basis of Equation (15). The public parameters fix the ZeroCheck protocol, GKR protocol, constrained-code construction, transcript, and opening protocol. None is selected or negotiated by proof bytes. A proof system without byte-exact ZeroCheck and GKR ledgers or without a canonical codeword construction and root vectors is incomplete and MUST NOT accept proofs.

For the deployed range 22 ≤ m ≤ 35, let J_m be the number of recursive opening levels. The level shapes and all parameters except the base fold difficulties are fixed by

$$\begin{aligned}
 h_i &:= m - 11 - 3i, & J_m &:= \min\{J \geq 1 : h_{J-1} \in \{3, 4, 5\}\}, \\
 r_i &:= 1 + i, & k_i &:= \begin{cases} 4, & i = 0, \\ 3, & i > 0, \end{cases} \\
 Q_i &:= (183, 90, 60, 45, 36, 31, 27, 24)_i, & g_i &:= 16, \\
 o_i &:= \begin{cases} 0, & i = 0, \\ 1, & i > 0. \end{cases}
 \end{aligned} \tag{71}$$

Here h_i is the message-column exponent, r_i the inverse-rate exponent, k_i the fold count, Q_i the query count, g_i the query-grinding difficulty, and o_i the OOD sample count. Implementations MUST use the integers in Equation (71), not a floating-point recomputation.

The remaining base fold difficulties f_i are the following literal table. Only the first J_m entries of the row for m exist.

m	$(f_0, \dots, f_{J_m-1})$
22	(9, 6, 3)
23	(10, 7, 4, 1)
24	(11, 8, 5, 2)
25	(12, 9, 6, 3)
26	(13, 10, 7, 4, 2)
27	(14, 11, 8, 5, 3)
28	(15, 12, 9, 6, 4)
29	(16, 13, 10, 7, 5, 3)
30	(17, 14, 11, 8, 6, 4)
31	(18, 15, 12, 9, 7, 5)
32	(19, 16, 13, 10, 8, 6, 3)
33	(20, 17, 14, 11, 9, 7, 4)
34	(21, 18, 15, 12, 10, 8, 5)
35	(22, 19, 16, 13, 11, 9, 6, 6)

6.2 Primitive canonical encodings

All unsigned integers are fixed-width little-endian. Decoders MUST NOT reduce, truncate, ignore high bits, or convert to a host-sized integer before checking that the value fits. The following encodings are the only canonical ones.

Type	Bytes	Canonicity rule
$u_8, u_{16}, u_{32}, u_{64}, u_{128}$	1, 2, 4, 8, 16	Unsigned little-endian of exactly the named width.
$a \in \mathbb{K}$	16	$\text{LE}_{64}(a_{10}) \parallel \text{LE}_{64}(a_{hi})$ in the fixed polynomial basis; coefficient bit j is bit j of the resulting 128-bit word.
$a \in \mathbb{F}_q$	16	$\text{LE}_{128}(\text{can}_q(a))$; reject an integer at least $q = 2^{100} - 15$.
Chunk exponent u	16	$\text{LE}_{128}(u)$; require $u < 2^{127}$.
Nonce	8	$\text{LE}_{64}(n)$.
Merkle value	32	Raw SHA-256 digest bytes, in digest output order.
Vector $[x_0, \dots, x_{n-1}]$	4 plus elements	$\text{LE}_{32}(n)$ followed by canonical elements in index order; n must equal the protocol-derived expected count.
Opened rows and path	nested	Vector of rows, each a vector of $\mathbb{K}$ elements, followed by a vector of 32-byte Merkle values. Expected row, width, and path counts are derived before allocation.

Every composite is concatenation without padding. Options, Rust enums, **usize**, varints, serde defaults, and bincode layouts are not wire types. For every accepted primitive or composite x , decoding its encoding must consume the whole bounded slice and re-encoding the result must reproduce the same bytes.

6.3 Merkle rows and canonical multiproofs

For opening level $0 \leq k < J_m$, define the message dimension, tree depth, leaf count, and row width by

$$n_k := h_k + k_k, \quad D_k := h_k + r_k, \quad \Lambda_k := 2^{D_k}, \quad I_k := 2^{k_k}. \quad (72)$$

Equation (72) implies $n_0 = m - 7$, $n_k = h_{k-1}$ for $k > 0$, $D_k = m - 10 - 2k$, $I_0 = 16$, and $I_k = 8$ for $k > 0$. Let $C_k \in \mathbb{K}^{\Lambda_k \times I_k}$ be the level- k codeword matrix, with leaves and lanes in increasing integer order. Its exact row and tree bytes are

$$\begin{aligned} \text{RowBytes}_k(q) &:= \text{Enc}_{\mathbb{K}}(C_k[q, 0]) \parallel \dots \parallel \text{Enc}_{\mathbb{K}}(C_k[q, I_k - 1]), \\ H_{k,0,q} &:= \text{SHA256}(\text{RowBytes}_k(q)), \\ H_{k,d+1,j} &:= \text{SHA256}(H_{k,d,2j} \parallel H_{k,d,2j+1}), \\ \text{root}_k &:= H_{k,D_k,0}. \end{aligned} \quad (73)$$

Here $\text{Enc}_{\mathbb{K}}$ is the 16-byte encoding in Section 6.2. Equation (73) uses no native-memory cast, row-length prefix, row index, level tag, padding, or leaf/internal-node domain byte. Adding such a byte changes every root and requires a new wire version. The fixed code specification defines how C_k is generated; this section fixes its portable row and tree encoding.

Let the already-derived query set be $A_0 = (q_0 < \dots < q_{Q_k-1})$. At depth d , scan the active positions A_d in increasing order. If both siblings are active, combine the pair once and emit no digest. Otherwise emit the digest at position $p \text{ xor } 1$, using p 's parity to place it on the left or right. Then set $A_{d+1} := \text{sortuniq}\{[p/2] : p \in A_d\}$. The canonical multiproof is the concatenation of emitted digests, bottom-up and in that scan order, and its exact count is

$$M_k := \sum_{d=0}^{D_k-1} |\{p \in A_d : (p \text{ xor } 1) \notin A_d\}|. \quad (74)$$

Equation (74) fixes the exact number of transmitted path digests. The level- k opening payload is exactly

$$\text{LE}_{32}(Q_k) \parallel \prod_{j=0}^{Q_k-1} (\text{LE}_{32}(I_k) \parallel \text{RowBytes}_k(q_j)) \parallel \text{LE}_{32}(M_k) \parallel \prod_{u=0}^{M_k-1} \text{path}_k[u], \quad (75)$$

with byte length

$$8 + Q_k(4 + 16I_k) + 32M_k. \quad (76)$$

The verifier derives A_d , M_k , and Equation (76) from the sampled queries before allocating or reading the bounded Hint record, requires rows in q_j order, reconstructs Equation (73), and rejects unused path digests or payload bytes.

6.4 Outer two-stream container

The 40-byte header is fixed as follows. Offsets and sizes are decimal; all multibyte integers use Section 6.2.

offset	size	field
0	8	magic = 46 32 5a 50 43 53 00 00
8	2	wire_version = 01 00
10	2	header_len = 28 00
12	4	flags = 00 00 00 00
16	4	narg_record_count
20	4	hint_record_count
24	8	narg_byte_len
32	8	hint_byte_len
40	...	NARG records, then Hint records

(40)

Writing Hdr_{40} for those exact header bytes, the only v1 host encoding is

$$\text{HostEncode}_{v1}(N, H) := \text{Hdr}_{40} \parallel N \parallel H, \quad \text{bytelen}(\text{HostEncode}_{v1}(N, H)) = 40 + \text{bytelen}(N) + \text{bytelen}(H). \quad (77)$$

Equation (77) implies the four benchmark metrics

$$\begin{aligned} \text{narglen}(\pi) &= \text{bytelen}(N), & \text{hintlen}(\pi) &= \text{bytelen}(H), \\ \text{payloadlen}(\pi) &= \text{bytelen}(N) + \text{bytelen}(H), & \text{wirelen}(\pi) &= 40 + \text{payloadlen}(\pi). \end{aligned} \quad (78)$$

The decoder MUST verify Equation (78) with checked arithmetic before allocating, enforce statement-derived upper bounds, and require input length exactly $40 + \text{narg_byte_len} + \text{hint_byte_len}$. The two declared record counts must match the records actually consumed. Header count and length fields are transport metadata and are not transcript input.

6.5 Typed event frames and registry

Every NARG record, Hint record, public absorb, and squeeze event uses the same prefix-free frame header:

offset	size	field	
0	2	event_tag	u16-le
2	2	reserved = 0	u16-le
4	4	chunk	u32-le
8	4	level	u32-le
12	4	round_or_index	u32-le
16	8	payload_len	u64-le
24	...	payload[payload_len]	

Let $\perp_{32} := 0\text{xffffffff}$ denote an inapplicable scope coordinate. For tag τ , scope (ℓ, k, i) , and payload z , the frame is exactly

$$\text{Frame}_{v1}(\tau, \ell, k, i; z) := \text{LE}_{16}(\tau) \parallel \text{LE}_{16}(0) \parallel \text{LE}_{32}(\ell) \parallel \text{LE}_{32}(k) \parallel \text{LE}_{32}(i) \parallel \text{LE}_{64}(\text{bytelen}(z)) \parallel z. \quad (79)$$

Thus Equation (79) has length $24 + \text{bytelen}(z)$. A stream record with a reserved nonzero word, an inapplicable coordinate other than $\perp_{32}$, a wrong channel, a wrong expected scope, or an unknown tag is malformed.

The base-tag registry is below. D events are public or derived, have no proof record, and are absorbed at their ledger position; N records occur only in N and are absorbed; H records occur only in H and are checked but never absorbed; S tags are squeeze requests. For an S base tag τ , the squeeze-result tag is $\tau \text{ OR } 0\text{x8000}$ and is derived, never transmitted. For an S row, the last column describes the derived result; its request payload is always $\text{LE}_{32}(n) \parallel a$ as fixed by Equation (83).

Tag	Class	Event	Scope; proof payload or squeeze result
0001	D	domain/version	$(\perp, \perp, \perp)$; DomainV1 from Equation (81).
0002	D	statement	$(\perp, \perp, \perp)$; StatementV1 from Equation (81).
0003	D	core seam	$(\perp, \perp, \perp)$; CoreSeamV1 from Equation (82).
1000-1fff	ZC	ZeroCheck	The fixed ZeroCheck protocol owns a disjoint, byte-exact ledger within this range.
2001	N	chunk ν	$(\ell, \perp, \perp)$; vector of 2^s elements of $\mathbb{K}$.
2081	H	chunk u	$(\ell, \perp, \perp)$; vector of 2^s canonical $u_{128} < 2^{127}$.
2101	S	chunk ζ	$(\ell, \perp, i)$ for $0 \leq i < s$; one $\mathbb{K}$ challenge.
3001	N	GKR phase-c sum	$(\ell, k, \perp)$; one $\mathbb{K}$ element, absorbed after the phase-link check and before the first phase-c round.
3002-30ff	GKR	GKR messages	The fixed GKR protocol owns exact message tags, codecs, and checks in this range.
3100-31ff	GKR	GKR squeezes	The fixed GKR protocol owns exact squeeze tags and result types in this range.
4001	N	ring s vector	$(\ell, \perp, \perp)$; vector of exactly 128 elements of $\mathbb{K}$.
4002	N	LinCheck round	$(\perp, \perp, i)$ for $0 \leq i < m$; exactly the three coefficients of $g_i(X) = \gamma_{i,0} + \gamma_{i,1}X + \gamma_{i,2}X^2$, encoded as $\text{Enc}_{\mathbb{K}}(\gamma_{i,0}) \parallel \text{Enc}_{\mathbb{K}}(\gamma_{i,1}) \parallel \text{Enc}_{\mathbb{K}}(\gamma_{i,2})$, with no vector prefix.
4101	S	shared r''	$(\perp, \perp, i)$ for $0 \leq i < 7$; one $\mathbb{K}$ challenge.
4102	S	batch η	$(\ell, \perp, \perp)$ in increasing active-chunk order; one $\mathbb{K}$ challenge.
4103	S	LinCheck challenge	$(\perp, \perp, i)$ for $0 \leq i < m$; one $\mathbb{K}$ challenge.
5001	D	opening target	$(\perp, \perp, \perp)$; $\text{LE}_{32}(m_p) \parallel \text{Enc}_{\mathbb{K}}(\beta)$. B is recomputed.
5002	N	initial root	$(\perp, 0, \perp)$; one 32-byte SHA-256 digest.
5003	N	recursive root	$(\perp, k, \perp)$, $1 \leq k < J_m$; one digest.
5004	N	sumcheck message	scope by kind below; kind: u_s followed by $\text{Enc}_{\mathbb{K}}(u_0) \parallel \text{Enc}_{\mathbb{K}}(u_2)$.
5005	N	OOD value	$(a, k, \perp)$; one $\mathbb{K}$ element for sample a .
5006	N	final y_r	$(\perp, J_m - 1, \perp)$; vector of exactly 2^{J_m-1} elements of $\mathbb{K}$.
5007	N	fold-PoW nonce	$(\perp, k, i)$; one u_{64} .
5008	N	query-PoW nonce	$(\perp, k, \perp)$; one u_{64} .
5080	H	initial opening	$(\perp, 0, \perp)$; Equation (75).
5081	H	recursive opening	$(\perp, k, \perp)$, $1 \leq k < J_m - 1$; Equation (75).
5082	H	final opening	$(\perp, J_m - 1, \perp)$; Equation (75).
5101	S	fold challenge	$(\perp, k, i)$; one $\mathbb{K}$ challenge.
5102	S	OOD point	(a, k, c) ; one $\mathbb{K}$ coordinate, with c increasing.
5103	S	glue challenge	$(\perp, k, i)$ as fixed below; one $\mathbb{K}$ challenge.
5104	S	PoW seed	$(\perp, k, i)$; 16 bytes; request auxiliary payload C .
5105	S	query draw	$(\perp, k, a)$; one $\mathbb{K}$ challenge for attempt a .
5106	S	query alpha	$(\perp, k, c)$; one $\mathbb{K}$ challenge.

The ZeroCheck and GKR ranges are black-box namespaces, not permission to use implementation-defined bytes. The fixed protocols publish ledgers with the same columns as this table. The fixed 3001 event resolves the PCS-level phase-c placement ambiguity in Gap G1; the rest of that gap remains GKR-protocol work and no protocol may reassign the tag. The v1 outer pre-sumcheck omits the redundant `presums[ell].claimed_sums` field and derives $[e_d^{(\ell)} + 1]$, so tag 3081 is reserved and MUST NOT occur.

6.6 Transcript transitions and replay

Model the transcript state as the byte string hashed by an unkeyed BLAKE3 hasher. It starts empty. The domain/version frame 0001 and statement frame 0002 are absorbed, in that order, before any proof-dependent squeeze. The source-statement digest is

$$d_{\text{src}} := \text{BLAKE3}_{256}(\text{hex}^{-1}(46325a2d5043532f6170702d73746174656d656e742f763100) \parallel \text{source_statement_bytes}). \quad (80)$$

For v1 byte compatibility, Equation (80) retains the legacy ASCII domain literal `F2Z-PCS/app-statement/v1` followed by one NUL byte. Changing that literal requires a new wire version. The two initial frame payloads are exactly

$$\begin{aligned} \text{DomainV1} &:= \text{hex}^{-1}(46325a5043530000) \parallel \text{LE}_{16}(1), \\ \text{StatementV1} &:= d_{\text{src}}[32] \parallel \text{LE}_{32}(t) \parallel \text{LE}_{32}(s) \parallel \text{LE}_{32}(W) \\ &\quad \parallel \text{Enc}_{\mathbb{K}}(g) \parallel \rho[32]. \end{aligned} \quad (81)$$

Thus DomainV1 and StatementV1 have 10 and 92 bytes, respectively. ZeroCheck must make the source-statement byte string in Equation (80) canonical.

The derived core-seam payload is

$$\text{CoreSeamV1} := \text{LE}_{32}(2^t) \parallel \prod_{b=0}^{2^t-1} \text{LE}_{128}(w_b) \parallel \text{LE}_{32}(2^s) \parallel \prod_{c=0}^{2^s-1} \text{LE}_{128}(\text{can}_q(w'_c)) \parallel \text{LE}_{128}(\text{can}_q(y)), \quad (82)$$

with exactly $24 + 16(2^t + 2^s)$ bytes. It is absorbed under tag 0003 after deterministic Bitify and before the first tag-2001 record.

For a canonical frame F , absorption appends the exact frame bytes. A squeeze under base tag $\tau < 0x8000$, scope (ℓ, k, i) , output length n , and auxiliary request payload a is the transition

$$\begin{aligned} R &:= \text{Frame}_{v1}(\tau, \ell, k, i; \text{LE}_{32}(n) \parallel a), \\ \text{tr}' &:= \text{tr} \parallel R, \\ z &:= \text{BLAKE3-XOF}(\text{tr}')[0 \dots n], \\ \text{tr}'' &:= \text{tr}' \parallel \text{Frame}_{v1}(\tau \text{ OR } 0x8000, \ell, k, i; z). \end{aligned} \quad (83)$$

Equation (83) uses a clone/finalize-XOF of the incremental hasher after R ; it does not reset or key the hasher. A $\mathbb{K}$ challenge has $n = 16$ and decodes by Section 6.2. A vector challenge is a sequence of scalar squeezes, not one long squeeze. Unless its registry row says otherwise, the scalars use consecutive i values and the auxiliary request payload a is empty. Tag 4102 instead uses $(\ell, \perp, \perp)$ for each active ℓ in increasing order. Tag 5103 uses its one-byte glue kind, and tag 5104 uses the PoW context C ; these are the only nonempty core auxiliary payloads.

On a normal NARG read, the verifier validates the expected tag, scope, payload length, and canonical payload, then immediately appends the complete frame from Equation (79). A PoW nonce is the sole deferred NARG read: the verifier parses its expected canonical frame, permits no intervening event, checks Equation (85), and appends the frame only on success. On a Hint read, it validates the same framing but never appends those bytes. It must complete the hint predicate before the next squeeze. In particular, it reads and absorbs 2001, then reads 2081 and verifies the exponent bounds and $g^u = \nu$ before the first 2101 event. NARG replay follows verifier-event order, never Rust structure-field order. Acceptance requires no deferred record or pending hint and exact logical and byte exhaustion of both streams and every nested slice.

Tags 4002 and 4103 are enabled only when the fixed core mode declares a general-coefficient LinCheck; the direct mode requires zero such events. The virtual mode alternates, for $i = 0, \dots, m-1$, one tag-4002 read-and-absorb, the check $g_i(0) + g_i(1) = \sigma_{i-1}$, and one tag-4103 squeeze at the same scope. No event may intervene within a round. It then reads and absorbs the singleton tag-4001 vector at scope $(0, \perp, \perp)$, derives $z = \sum_v \text{eq}(v, r_{10})s_v$, and checks $\sigma_{m-1} = \bar{a}(r)z$ before tag 4101. The coefficient evaluator is verifier-derived and contributes no payload. This is the only LinCheck in the virtual path; the same tag-4001 vector continues into the inherited ring-switch computation.

6.7 Grinding and query derivation

Let kind byte zero denote a fold grind and kind byte one a query grind. For kind K_{pow} , level k , round i , and derived difficulty d , define the exact PoW context

$$C := \text{ASCII}(\text{F2Z-PCS/PoW/v1}) \parallel \text{LE}_8(K_{\text{pow}}) \parallel \text{LE}_{32}(k) \parallel \text{LE}_{32}(i) \parallel \text{LE}_{16}(d). \quad (84)$$

In Equation (84), $i = \perp_{32}$ for a query grind. The seed is the 16-byte output of Equation (83) under tag 5104, the same scope, and auxiliary request payload C . After obtaining the seed, the verifier reads the expected 5007 or 5008 NARG frame without absorbing it and requires, for the scheduled positive difficulty d ,

$$\text{LZ}(\text{BLAKE3}_{256}(C \parallel \text{seed}_{16} \parallel \text{LE}_{64}(n))) \geq d, \quad d > 0. \quad (85)$$

LZ in Equation (85) counts zero bits from the most-significant bit of digest byte zero, then proceeds toward the least-significant bit and subsequent bytes. Any satisfying nonce is accepted; minimality is not checked. Only after the predicate succeeds does the verifier absorb the complete nonce frame, before any dependent squeeze. Difficulty zero has no predicate invocation, seed squeeze, or nonce record in this wire format.

The exact scheduled difficulties are

$$d_k^{\text{query}} := g_k = 16, \quad d_{k,i}^{\text{fold}} := \max(f_k - i, 0). \quad (86)$$

Equation (86) is evaluated with ordinary nonnegative integers. After the preceding sumcheck message and immediately before fold challenge (k, i) , perform both the tag-5104 seed squeeze and tag-5007 nonce record if $d_{k,i}^{\text{fold}} > 0$; if the difficulty is zero, perform neither event. For $0 \leq k < J_m - 1$, query event k occurs after the tag-5003 commitment root for level $k+1$, all o_{k+1} OOD bindings for that root, and their glue challenges, and immediately before the tag-5105 draws that open level k . Query event $J_m - 1$ instead occurs after the tag-5006 final y_r , with no new root or OOD binding, and immediately before the final-level draws. There is exactly one query nonce at every level. Consequently the exact number of nonce records is

$$N_{\text{pow}} = J_m + \sum_{k=0}^{J_m-1} \min(k_k, f_k). \quad (87)$$

Missing, extra, reordered, or trailing nonces violate Equation (87) and reject.

For level k , each 5105 event yields $r \in \mathbb{K}$ and proposes

$$q := r_{10} \bmod 2^{h_k + r_k}. \quad (88)$$

The verifier repeats Equation (88) until Q_k distinct positions exist, discarding duplicates, and then sorts the positions in increasing order. It then derives the tag-5106 alphas before reading and checking the opening.

6.8 Recursive-opening replay ledger

This subsection fixes the complete replay order for the recursive opening. The four canonical tag-5004 message kinds and scopes are

0x00	Start	$(\perp, 0, \perp)$	exactly once,	
0x01	FoldNext	$(\perp, k, j)$	$0 \leq j < k_k$,	
0x02	OodIntro	$(a, k, \perp)$	$1 \leq k < J_m, 0 \leq a < o_k$,	
0x03	QueryIntro	$(\perp, k, \perp)$	$0 \leq k < J_m - 1$.	(89)

Equation (89) is the complete tag-5004 kind registry. Each payload is the one-byte kind followed by the two canonical $\mathbb{K}$ elements in the registry. Any other kind/scope pair rejects.

The outer **VerifyF2Z** caller first requires that B is the deterministic coefficient function derived in P10/V10, with domain size $2^m p$; B is never densely serialized. The caller alone absorbs tag 5001 with payload $\text{LE}_{32}(m_p) \parallel \text{Enc}_{\mathbb{K}}(\beta)$ and then invokes **RecVerifyStream**. That helper starts after tag 5001: its first event reads tag 5002, requires the root to equal ρ , absorbs it, and then reads and absorbs the Start tag-5004 message.

For each $k = 0, \dots, J_m - 1$ in increasing order, it then performs the following ledger without interleaving another event.

1. For $j = 0, \dots, k_k - 1$, perform the optional fold grind exactly as in Section 6.7, squeeze tag 5101 at $(\perp, k, j)$, then read, validate, and absorb the FoldNext tag-5004 message at the same scope.
2. If $k < J_m - 1$, read and absorb the tag-5003 root at $(\perp, k+1, \perp)$. For each $a = 0, \dots, o_{k+1} - 1$, squeeze the n_{k+1} tag-5102 coordinates at scopes $(a, k+1, c)$ for $c = 0, \dots, n_{k+1} - 1$; read and absorb tag 5005 at $(a, k+1, \perp)$; read and absorb the OodIntro tag-5004 message at that scope; and squeeze tag 5103 at $(\perp, k+1, a)$ with the one-byte auxiliary kind 00.
3. If instead $k = J_m - 1$, read and absorb tag 5006 at $(\perp, k, \perp)$, requiring exactly 2^{h_k} canonical $\mathbb{K}$ elements.
3. Perform the query grind at $(\perp, k, \perp)$ with difficulty 16 and absorb its validated tag-5008 nonce. Starting with attempt $a = 0$, squeeze tag 5105 at $(\perp, k, a)$ and increment a after every squeeze, including a duplicate, until Q_k distinct positions have been obtained. Reject rather than wrap if the next attempt would exceed $2^{32} - 1$. Sort the positions, then squeeze exactly $\lceil \log_2 Q_k \rceil$ tag-5106 alphas at $(\perp, k, c)$ for consecutive c .

Read exactly one opening Hint using tag 5080 for $k = 0$, tag 5081 for $0 < k < J_m - 1$, or tag 5082 for $k = J_m - 1$. Check Equations (75)–(76) and the root before any later event. For $k < J_m - 1$, next read and absorb the `QueryIntro` tag-5004 message. At every level, including the final level where no such message exists, squeeze the query glue tag 5103 at $(\perp, k, o_k)$ with one-byte auxiliary kind 01. At the final level this challenge is used directly in the residual check.

The fixed record/event cardinalities are therefore

$$\begin{aligned}
 \#5002 &= 1, & \#5003 &= J_m - 1, & \#5005 &= \sum_{k=1}^{J_m-1} o_k, \\
 \#5004 &= 1 + \sum_{k=0}^{J_m-1} k_k + \sum_{k=1}^{J_m-1} o_k + (J_m - 1), & \#5006 &= 1, & \#508* &= J_m, \\
 \#5101 &= \sum_{k=0}^{J_m-1} k_k, & \#5102 &= \sum_{k=1}^{J_m-1} o_k n_k, & \#5103 &= J_m + \sum_{k=1}^{J_m-1} o_k, \\
 \#5106 &= \sum_{k=0}^{J_m-1} \lceil \log_2 Q_k \rceil.
 \end{aligned} \tag{90}$$

The cardinalities in Equation (90) are exact. Here #508* is one opening Hint per level. Tag-5105 count is the transcript-derived number of attempts, all of which appear in the audit log. Nonce counts remain Equation (87). A different order, scope, kind, cardinality, or unread field rejects.

6.9 Parser, implementation, and conformance requirements

A v1 implementation **MUST**:

1. validate magic, version, flags, all public-parameter-, statement-, and transcript-derived bounds, and all reserved fields before allocating or doing proof work;
2. use checked arithmetic and checked conversion for every length and index, reject noncanonical field values, and return a decoding error rather than panic on adversarial bytes;
3. encode the NARG stream in the exact absorb order and the Hint stream in its exact check order, with independent cursors;
4. replace opaque recursive-proof serialization with the field records and exact order in Sections 6.5 and 6.8; and
5. reject unless every outer, subordinate, recursive, row, and Merkle-path slice is exactly exhausted.

Conformance fixtures **MUST** include canonical primitive and frame hex, one complete $N/H/\text{HostEncode}_{v1}$ proof, the ordered transcript audit log, every squeezed challenge, PoW seeds/nonces and query positions, and all four lengths from Equation (78). Negative fixtures **MUST** cover an unknown version or tag, a reserved bit, wrong endian or channel, record reordering, truncation and suffix bytes, noncanonical scalars, count overflow, a failed or late u/ν hint, altered PoW scope/difficulty/nonce, and missing or extra recursive fields. A LinCheck-enabled fixture must additionally cover a wrong round polynomial, reordered 4002/4103 pair, altered terminal coefficient, and failed terminal gate. Native, recursive, and independent reference verifiers must produce byte-identical transcript audit logs.

The repository file `wire-v1-vectors.txt` is the normative initial known-answer set for the primitive container/frame codec, Merkle row/tree/multiproof codec, and PoW predicate at difficulties 1, 7, 8, and 16. A conforming implementation must reproduce it exactly and add a complete proof/transcript fixture before claiming end-to-end v1 interoperability.